面向障碍环境多目标点遍历路径规划的协同优化框架

作者姓名 *Member, IEEE*

摘要—自主导航要求移动机器人在障碍物环境中以最优顺序访问一系列目标点，这一多目标点遍历路径规划问题在无人公交接驳、物流配送和基础设施巡检等场景中至关重要。该问题同时耦合两个子问题：一是在杂乱环境中寻找点对之间的无碰撞路径，二是确定目标点的最优访问顺序。现有研究往往只处理其中一个子问题——旅行商求解器默认欧氏距离，路径规划器则只优化单对点之间的路由。针对这一空白，本文提出一种将两者统一的协同优化框架。首先，基于跳点搜索（**Jump Point Search, JPS**）的改进 **A*** 全局规划器将障碍信息编码为旅行商代价矩阵，**JPS** 利用栅格对称性跳过冗余节点扩展；随后结合安全感知的路径简化（拐点提取、贪心扫描与中间点探索）以及弧长参数化三次样条平滑。所得开放旅行商问题（起点与终点固定且不重合）由一种改进蚁群优化算法求解：通过虚拟节点法将其转化为闭环旅行商问题，并融合最大-最小蚁群系统的信息素管理、蚁群系统的伪随机转移规则、候选列表加速的可变邻域下降局部搜索，以及 **S** 形渐进调度和自适应停止准则。在三个测试环境上的实验表明，基于 **JPS** 的规划器相较 **Dijkstra** 最多减少约 **99%** 的节点扩展、搜索速度提升约 **17** 倍，并生成更短的路径；参数敏感性分析确定了一组均衡默认配置；消融实验表明可变邻域下降局部搜索与最大-最小蚁群系统信息素边界是保证求解质量的两个关键组件；所提出的求解器在 **kroB100** 基准上以 **97.5%** 的命中率达到最优代价，显著优于标准蚁群优化、遗传算法和模拟退火，并表现出更高的稳定性。

Index Terms—多目标点遍历，路径规划，跳点搜索，蚁群优化，开放旅行商问题，避障，虚拟节点法

I. 引言

多目标点遍历路径规划是自主移动机器人领域的一项基本问题，在无人公交接驳、物流仓储管理、基础设施巡检以及搜救行动等场景中具有广泛应用 [1], [2]。在完成例如接送乘客、检查设备、投递物品等特定任务中，机器人需要从指定起始位置出发，依次访问一系列中间目标点，最终到达目标终点。规划器需解决两个既独立又耦合的子问题：

本文作者单位与通讯信息待补充。

稿件收稿日期：2026 年；修订日期：2026 年。

（1）在杂乱环境中寻找连续点对之间的无碰撞路径；

（2）确定中间目标点的最优访问顺序。第一个子问题属于几何问题——需要在物理空间中对障碍物几何形状和路径连续性进行推理。第二个子问题属于组合优化问题——是经典 NP-hard 旅行商问题（TSP）的一个实例。这两个子问题相互影响，任意两点之间的通行代价取决于其间存在的障碍物，而最优访问顺序又取决于这些点对间的通行代价。

总体来看，现有多目标导航方法通常仅独立处理其中一个子问题。大量旅行商问题研究假设点对间的代价由欧氏距离或已知距离矩阵给出 [3], [4]，忽略了在障碍物丰富的环境，两点之间的最短无碰撞路径可能远长于直线距离这一事实。另一方面，例如 A*、Dijkstra、快速扩展随机树（RRT）及其变体路径规划领域的成熟算法能够高效地计算栅格地图中点对之间的最优或近似最优无碰撞路径，但这些规划器并不优化多目标点的访问顺序 [5]–[7]。少量研究在特定几何假设下探索了带障碍物的旅行商问题变体（如带障碍物的 TSP、Dubins TSP），但目前尚缺乏一种将基于栅格的全局规划器与高性能旅行商求解器相集成的、通用的障碍环境多目标点遍历框架。

针对上述约束并最大化利用两类算法的优势，本文旨在提出一种面向障碍环境多目标点遍历路径规划的协同优化框架。该框架由两个算法层组成，通过预计算的代价矩阵连接。第一层是基于跳点搜索（JPS）的改进 A* 全局规划器，其利用栅格对称性跳过冗余节点扩展、仅扩展跳点，从而大幅缩减搜索树规模；该规划器结合了安全感知的路径简化流程（含拐点提取和中间点探索）以及弧长参数化三次样条平滑管线。第二层是面向开放旅行商问题的改进蚁群优化（ACO）求解器，其通过虚拟节点（哑节点）法将开放旅行商问题转化为标

准闭环旅行商问题，从而充分利用闭环旅行商问题的搜索算子；该求解器融合了最大-最小蚁群系统（MMAS）的信息素管理、蚁群系统（ACS）风格的伪随机转移规则、以 k 近邻候选列表加速的可变邻域下降（VND）局部搜索引擎，以及基于 S 形函数的渐进式局部搜索调度和自适应停止准则。

本文的贡献分为三个层面：

1) 在问题层面，将障碍环境多目标点遍历形式化为一个集成优化任务，并提出通过全局规划器把障碍信息编码为旅行商代价矩阵的两层框架，从而弥合孤立路径规划与无约束旅行商研究之间的空白，使其适用于存在障碍物且起点、终点不重合的真实导航任务。
2) 在算法协同层面，引入具有安全检测的真实路径代价计算机制，在评估代价矩阵元素之前应用安全感知的路径规划运算，使旅行商代价函数更准确地反映真实连续路径长度，从而提升求解器区分不同访问顺序方案的能力；同时，模块化架构允许将任意栅格规划器与旅行商求解器替换到相应层中。
3) 在组件层面，路径规划方面采用跳点搜索剪枝冗余节点，并以安全感知路径简化生成最小安全航点子集；旅行商优化方面采用虚拟节点法使起点、终点完整参与信息素模型与局部搜索，以候选列表加速的可变邻域下降将局部搜索复杂度从 $O(K^2)$ 降至 $O(K \cdot k)$，并以 S 形渐进调度与自适应停止机制平衡探索、利用并节省计算。上述组件均通过消融实验得到独立验证。

本文其余部分组织如下：第 II 节综述全局路径规划、旅行商元启发式方法及带障碍物多目标点遍历的相关研究；第 III 节形式化问题定义；第 IV 节详述方法论，包括改进的 A* 规划器（第 IV-A 节）、路径后处理管线（第 IV-B 节）、改进的蚁群优化求解器（第 IV-C 节）和协同优化框架（第 IV-D 节）；第 V 节报告四个评估维度的实验结果：全局路径规划对比、参数敏感性分析、消融实验以及不同旅行商求解器的多目标点遍历对比；第 VI 节总结全文并展望未来工作。

II. 相关工作

A. 栅格环境中的全局路径规划

全局路径规划旨在已知静态环境中寻找两点之间的无碰撞路径。经典图搜索算法仍是栅格规划的主流方法。Dijkstra 算法保证最短路径，但以均匀方式向所有方向扩展，在大地图上效率较低 [6]。A* 算法通过引入启发式函数 $h(n)$ 引导搜索方向，同时在 $h(n)$ 可采纳时保持最优性 [5]。然而，标准 A* 使用固定的启发式权重，面临权衡困境：权重为 1 时保证最优性但扩展节点过多，较大的常数权重可加速搜索但牺牲路径最优性。若干 A* 变体针对此局限进行了改进 [8]。加权 A* 将启发式乘以常数因子 $\varepsilon > 1$，以更快的速度产生 ε-最优解 [9]。任意时间修复 A* 及其后续方法交替进行加权和无加权搜索，生成逐步改进的解序列 [10]。然而，这些方法在整个搜索过程中施加统一权重，而最优权重直观上取决于到目标的距离：远离目标时激进权重有利，接近目标时需要精度。跳点搜索（JPS）利用栅格对称性跳过冗余节点，仅扩展最优路径改变方向或遇到强制选择的跳点 [11], [12]。我们的基于 JPS 的规划器（第 IV-A 节）采纳了这一原理，近期的 JPS 变体进一步通过双向搜索和任意角度路径精化提升了效率 [13]–[15]。

基于采样的规划器如快速扩展随机树（RRT）[7] 和概率路线图（PRM）提供概率完备性，在高维构型空间中表现良好，但其在栅格环境中的解路径通常比图搜索方法更长、一致性更差，这是由于随机采样的固有特性所致。

栅格路径的后处理是成熟的技术实践。路径裁剪方法通过共线性检查或视线验证移除冗余共线航点 [16]。路径平滑方法包括三次样条、B 样条和回旋曲线等，将离散航点序列转化为适合机器人执行的连续曲线 [17], [18]。然而，裁剪和平滑通常被视为独立的事后步骤；将它们集成到代价矩阵计算中——即用简化路径长度指导组合优化——是我们框架的一个显著特征。

B. 旅行商问题的元启发式方法

旅行商问题是组合优化领域中研究最为深入的问题之一。基于分支定界或动态规划的精确算法仅适用于小规模实例（通常 $K \leq 20$），这源于搜索空间的阶乘级增长。对于更大规模的实例，元启发式方法是首选方案。

遗传算法（GA）将路径编码为染色体，应用交叉算子（如顺序交叉 OX）和变异算子，通过锦标赛或轮盘选择挑选最优个体 [19]。模拟退火（SA）通过邻域移动（通常为 2-opt 交换）探索解空间，并以随退火计划衰减的概率接受劣化解，从而逃脱局部最优 [20]。

蚁群优化（ACO）是一种受蚂蚁觅食行为启发的种群元启发式方法。在基本蚁群系统中，蚂蚁根据信息素

轨迹和启发式可见度概率性地构建路径，并在其路径上沉积信息素以引导后续世代 [21]。蚁群系统通过伪随机比例转移规则扩展了基本蚁群系统，在贪心利用与概率探索之间取得平衡，提高了收敛速度 [22]。最大-最小蚁群系统通过施加信息素值的显式上下界进一步稳定搜索过程，防止对次优解的过早收敛，同时在低频边上保持探索多样性 [23]。

局部搜索混合化在蚁群优化中已被证明非常有效。对每一代最优解应用 2-opt、3-opt 或可变邻域下降（VND）可显著提升解质量 [24], [25]。然而，VND 对每个邻域算子评估 $O(K^2)$ 个候选移动，在 $K > 100$ 时成为计算瓶颈。候选列表技术通过将评估限制于节点的最近邻来加速局部搜索，已被广泛采用 [26]。我们的候选列表加速 VND（第 IV-C4 节）通过预计算的 k 近邻列表剪枝无前景的评估，将复杂度降至 $O(K \cdot k)$，使大规模实例的局部搜索变得可行。

渐进式局部搜索调度在文献中受到的关注较少。大多数蚁群优化实现在整个运行过程中对固定比例的蚂蚁施加局部搜索。近期的蚁群优化变体强调并行化、小生境化和面向可持续性的机制，以应对大规模实例 [27]–[30]。我们的 S 形调度（第 IV-C5 节）在迭代预算内将局部搜索比例从零递增至最大值，匹配了从早期探索主导到后期精化主导的自然转变。

C. 带障碍约束的多目标点遍历

路径规划与旅行商优化的交叉领域已在多个名义下得到探索 [4]。带障碍物的旅行商问题（TSPO）考虑连续平面中的一组多边形障碍物，寻求访问所有目标点且不与障碍物相交的最短路径 [3]。其解通常通过计算可见图并在所得图上求解旅行商问题。Dubins 旅行商问题将问题扩展至具有最小转弯半径约束的车辆。邻域旅行商问题将目标从点泛化为区域，要求路径至少访问每个区域一次。这些公式依赖于连续几何表示（多边形、圆盘）并假设特定的障碍形状，限制了其在基于栅格的机器人导航中的适用性——在栅格导航中，障碍物以任意占用模式表示。

结合路径规划器与旅行商求解器的两层架构已有若干先例 [1], [2]。例如，概率路线图被用于构建连通图，并在其上以采样构型间的最短路径距离求解旅行商问题。然而，这些方法通常采用现成的规划器和求解器，未针对多目标点遍历上下文优化任一层。代价矩阵由原始栅格路径计算得出，因离散化伪影而高估了真实路径长度，且旅行商求解器缺乏我们框架所提供的问题特定增强（如开放旅行商编码、候选列表加速）。

开放旅行商变体——其中起点和终点为不重合的固定位置——与自主导航任务尤为相关，机器人从仓库出发并到达目的地。经典旅行商问题文献的大部分内容，包括大多数 TSPLIB 基准实例，针对的是闭环旅行商问题。开放旅行商问题需要不同的编码和代价评估函数，我们在第 III-B 节中将其形式化，并在第 IV-C1 节中通过虚拟节点（哑仓库）法给出算法实现，该方法类似于开放车辆路径问题中所采用的技术 [31], [32]。

综上，A* 类方法可给出高质量的点对点路径，开放式旅行商元启发式则擅长组合排序，二者单独使用均难以覆盖障碍环境下的完整多目标点遍历任务。将基于跳点搜索的全局规划、安全感知后处理与开放旅行商蚁群优化相集成，可在保留几何可行性的同时提升访问顺序优化的质量。据我们所知，尚无先前工作同时满足以下三点：（1）基于跳点搜索的改进栅格全局规划器与安全感知路径后处理；（2）融合候选列表加速 VND 和 S 形渐进局部搜索的开放旅行商蚁群优化求解器；（3）利用简化路径代价实现精确组合优化的协同代价矩阵接口。因此，本文提出了相应的协同优化框架。

III. 问题形式化

本节将障碍环境多目标点遍历路径规划问题进行形式化定义。首先建立基于栅格的环境模型，然后定义具有固定起点和终点的开放旅行商问题，最后给出完整的优化目标。

A. 栅格地图与障碍物表示

运行环境建模为 $n \times n$ 单元的二维方形栅格地图。每个栅格单元被分类为自由空间或障碍物：

$$\mathbf{M}(r,c) = \begin{cases} 0, & \text{栅格}(r,c)\text{为自由空间} \\ 1, & \text{栅格}(r,c)\text{被占用} \end{cases} \tag{1}$$

其中 $r \in \{1,\dots,n\}$ 和 $c \in \{1,\dots,n\}$ 分别为行索引和列索引。假设地图在规划前静态且完全已知。

相邻自由栅格之间的移动遵循 8 邻域连通性：从栅格 (r,c) 可向其周围 8 个相邻栅格 $(r+\Delta r, c+\Delta c)$ 移动，其中 $\Delta r, \Delta c \in \{-1,0,1\}$，$(\Delta r, \Delta c) \neq (0,0)$，前提是目标栅格为自由空间。移动代价区分正交步和对角步：

$$\mathrm{cost}(\Delta r,\Delta c)=\begin{cases}1, & |\Delta r|+|\Delta c|=1\text{（正交步）}\\ \sqrt{2}, & |\Delta r|+|\Delta c|=2\text{（对角步）}\end{cases} \tag{2}$$

对于对角移动，相邻的两个正交栅格也必须同时为自由空间；否则机器人将穿越障碍物角点，这对非点状机器人来说在物理上不可行。

栅格坐标 (r,c) 通过将栅格中心置于其几何中心处映射为连续世界坐标 (x,y)：

$$x=c-0.5,\quad y=r-0.5 \tag{3}$$

此映射将连续坐标系的原点置于栅格左下角（栅格 $(1,1)$ 映射为 $(0.5,0.5)$），与占用栅格地图中机器人定位的标准约定一致。

B. 具有固定起点和终点的开放旅行商问题

设任务规格包含一个起点 S、K 个中间目标点 $\mathcal{T}=\{T_1,T_2,\dots,T_K\}$ 和一个终点 G。所有点以栅格坐标 $[r,c]$ 指定。完整点集定义为：

$$\mathcal{P}=\{p_1,p_2,\dots,p_N\},\quad N=K+2 \tag{4}$$

索引约定为：$p_1=S$，$p_i=T_{i-1}$（$i=2,\dots,K+1$），$p_N=G$。起点和终点不重合，占据任何可行解的边界位置。

一个有效路径由访问顺序 $\boldsymbol{\pi}=(\pi_1,\pi_2,\dots,\pi_N)$ 定义，该顺序是索引集 $\{1,2,\dots,N\}$ 的排列，满足边界约束：

$$\pi_1=1\ \text{（起点）},\qquad \pi_N=N\ \text{（终点）} \tag{5}$$

中间索引 $\{\pi_2,\dots,\pi_{N-1}\}$ 构成 $\{2,\dots,N-1\}$ 的一个排列，表示目标点的有序访问。可行路径的总数为 $K!$，对应中间目标点的所有排列且端点固定。这是**开放旅行商问题**的形式化，区别于经典闭环旅行商问题——后者的路径构成环路且起始节点可任意选取 [3], [4]。

C. 优化目标

对于任意有序点对 (p_i,p_j)，设 $\mathcal{P}(p_i,p_j)$ 表示从 p_i 到 p_j 在栅格地图 $\mathbf{M}$ 中的最短无碰撞路径，由全局路径规划器计算。该路径为栅格单元序列：

$$\begin{aligned}\mathcal{P}(p_i,p_j)=[p_i=q_1,q_2,\dots,q_m=p_j],\\ q_k\in\{(r,c)\mid\mathbf{M}(r,c)=0\}\end{aligned} \tag{6}$$

路径长度为序列中各边代价之和：

$$L\big(\mathcal{P}(p_i,p_j)\big)=\sum_{k=1}^{m-1}\mathrm{cost}(q_{k+1}-q_k) \tag{7}$$

其中 $\mathrm{cost}(\cdot)$ 由式 (2)定义。若 p_i 和 p_j 之间不存在无碰撞路径，则 $\mathcal{P}(p_i,p_j)=\varnothing$，$L(\varnothing)=\infty$。

路径 $\boldsymbol{\pi}$ 的总代价为 $N-1$ 段连续路径长度之和：

$$C(\boldsymbol{\pi})=\sum_{k=1}^{N-1}L\big(\mathcal{P}(p_{\pi_k},p_{\pi_{k+1}})\big) \tag{8}$$

障碍环境多目标点遍历问题表述为：

$$\begin{aligned}&\boldsymbol{\pi}^*=\arg\min_{\boldsymbol{\pi}\in\Pi}\ C(\boldsymbol{\pi})\\ &\text{s.t.}\quad \pi_1=1,\ \pi_N=N,\ \{\pi_2,\dots,\pi_{N-1}\}=\{2,\dots,N-1\}\\ &\qquad \mathcal{P}(p_{\pi_k},p_{\pi_{k+1}})\neq\varnothing\quad\forall k\in\{1,\dots,N-1\}\end{aligned} \tag{9}$$

其中 Π 为所有满足边界约束的 $\{1,\dots,N\}$ 排列的集合。式 (9)中的优化耦合了两个相互作用的子问题：（1）为每段路径寻找最短无碰撞路径（几何问题，由全局规划器求解）；（2）确定中间目标点的最优排序（组合问题，由旅行商求解器求解）。代价矩阵元素 $\mathbf{D}(i,j)=L(\mathcal{P}(p_i,p_j))$ 构成两个子问题之间的接口，将完整的障碍约束距离拓扑编码为 $N\times N$ 矩阵。该解耦是精确的：对于给定的代价矩阵 $\mathbf{D}$，式 (9)的优化退化为开放旅行商问题；对于给定的访问顺序 $\boldsymbol{\pi}$，每段路径是独立的全局规划问题。

在特殊情况 $K=0$（无中间目标点）下，问题退化为单对最短路径查询：$\boldsymbol{\pi}^*=(1,N)$，$C(\boldsymbol{\pi}^*)=L(\mathcal{P}(S,G))$，即经典单对路径规划问题。

IV. 方法论

为克服单一路径规划器无法优化访问顺序、以及无约束旅行商模型忽略障碍代价的局限，本节提出一种由两个阶段组成的启发式协同框架。本章算法中的超参数均以符号给出，除特别说明外默认取值统一列于表 I。具体而言，第一阶段采用本文提出的改进 A* 全局规划器在栅格地图上搜索点对点无碰撞最短路径，并通过安全距离感知的路径简化与三次样条平滑生成高

质量连续路径；第二阶段以该阶段输出的代价矩阵为输入，由本文提出的 MMAS-VND-CL 蚁群优化求解器求解多目标点的最优访问序列。两阶段通过代价矩阵实现耦合，共同完成从几何寻径到组合排序的完整流程。第 IV-A 节和第 IV-B 节介绍全局路径规划及后处理，第 IV-C 节详述开放旅行商求解算法，第 IV-D 节阐述两层协同框架的整体设计。

A. 改进的 A 全局路径规划器*

*1) 跳点搜索：*传统 A* 算法在搜索过程中需要扩展大量节点，尤其在大规模开阔区域中，节点扩展数量与地图面积成正比，计算效率受到严重制约。为解决这一问题，本文提出基于跳跃点搜索（Jump Point Search, JPS）的改进 A* 算法 [11], [12]。JPS 的核心思想是利用栅格地图的对称性，在搜索过程中跳过大量冗余的中间节点，仅保留具有“跳跃意义”的关键节点作为扩展候选。

在传统 A* 中，当前节点的所有邻居均需加入 open set 进行评估，而 JPS 通过邻居剪枝大幅减少需要考虑的邻居数量。具体而言，剪枝策略取决于当前节点的扩展方向：

1) **直线移动：**仅保留自然邻居（移动方向前方的节点）和被迫邻居（因障碍物阻挡而被迫改变方向的节点）。对于直线移动方向两侧的节点，除非被障碍物阻挡产生被迫邻居，否则均被剪枝。
2) **对角线移动：**保留自然邻居（对角方向前方）、直线方向的自然邻居（水平和垂直分量方向），以及因障碍物产生的被迫邻居。

被迫邻居的判定条件为：当前节点沿移动方向的相邻位置被障碍物阻挡，但沿另一方向可达。图 1给出了剪枝规则的示意图。

进一步地，JPS 的跳跃函数 $\mathrm{jump}(r, c, dr, dc)$ 从当前节点 (r, c) 出发，沿方向 (dr, dc) 逐步前进，直到满足以下三个终止条件之一：（1）到达目标节点；（2）遇到障碍物或地图边界（该方向不可通行）；（3）发现被迫邻居（当前位置因障碍物阻挡而需要强制转向）。跳跃过程中经过的中间节点不会被加入 open set，从而显著减少了节点扩展数量。当遇到被迫邻居时，当前节点即为跳跃点，需要加入 open set 进行后续评估。对于对角线方向的跳跃，算法会先沿对角线前进一步，然后分别沿水平和垂直分量方向执行直线跳跃。若任一方向的直线跳跃发现了被迫邻居或到达目标，则返回当前对角线位置作为跳跃点。当 jump 函数返回可达的下一个节点后，还需要计算从当前节点到该跳跃点的实际移动成本。对于直线跳跃，成本等于跳跃的曼哈顿距离；对于对角线跳跃，成本等于对角线距离（每步对角线移动成本为 $\sqrt{2}$）。算法 1给出了 JPS 搜索过程的伪代码。路径重建方面，JPS 搜索完成后，通过父节点指针从目标节点回溯至起点，并在相邻跳跃点之间沿直线插值补充被跳过的中间栅格，即可得到完整路径。然而，由于 JPS 的跳跃特性，重建路径中的对角线移动可能穿越障碍物拐角。为此，本文算法在路径重建阶段引入了对角拐点修正机制：当检测到对角线移动穿越障碍物角时，将其替换为沿栅格边界行走的 L 形路径（先走行再走列，或先走列再走行），从而避免与障碍物发生碰撞。修正后的路径严格沿栅格边界行进，保证了路径的可行性。此外，JPS 的剪枝规则依赖于均匀的栅格地图结构。在包含复杂障碍物分布的地图中，被迫邻居的产生频率增加，JPS 的优势会被削弱，个别情况下对称性假设失效可能导致 JPS 搜索失败。为此，本文提出的改进 A* 算法在 JPS 搜索耗尽开放集仍未找到路径时，自动回退至传统 A* 重新搜索，以确保在存在可行解时总能找到路径。这一降级机制保证了算法的鲁棒性，使其能够适应不同复杂度的环境。

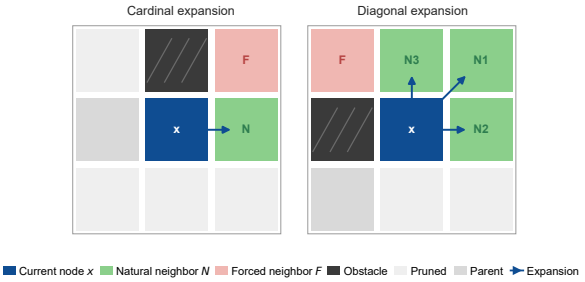


图 1. JPS 邻居剪枝规则示意图。

*2) 打破平局策略：*当开放集中的多个节点具有相同的 f 值时，标准 A* 的选择顺序取决于具体实现（通常是先入先出或随机），本质上是任意的。然而，在 f 值相同的节点中，h 值较小的节点意味着其实际代价 $g = f - h$ 较大，即该节点距起点更远、更接近目标。优先扩展此类节点可将搜索方向导向目标，且不影响最优性保证。为此，本文提出的改进 A* 算法采用“f 值相同时优先选择 h 值更小的节点”的打破平局策略，二叉

Algorithm 1 JPS-based A* search

Require: occupancy grid map, start grid S, goal grid G

Ensure: grid path $P = [S, \dots, G]$, or failure

initialize $g(S) = 0$, $f(S) = h(S, G)$, and insert S into the open set (a binary min-heap);
while the open set is not empty **do**
  $n \leftarrow$ the node in the open set with the smallest f-value, breaking ties by the smaller h;
  if $n = G$ **then**
    reconstruct and return the grid path P;
  end if
  move n into the closed set, and record its parent direction $\mathbf{d}_n$;
  $D \leftarrow$ the pruned successor directions of n determined by $\mathbf{d}_n$ (all eight directions if $n = S$);
  for each direction $\mathbf{d} \in D$ **do**
    $s \leftarrow \mathrm{Jump}(n, \mathbf{d})$, the first jump point encountered along $\mathbf{d}$, or null;
    if $s =$ null or s is closed **then**
      continue;
    end if
    tent_g $\leftarrow g(n) + \mathrm{Cost}(n, s)$, where straight steps cost 1 and diagonal steps cost $\sqrt{2}$;
    if tent_g $< g(s)$ **then**
      set the parent of s to n, $g(s) \leftarrow$ tent_g, $f(s) \leftarrow$ tent_g $+ h(s, G)$;
      insert s into the open set, or update its heap position if already present;
    end if
  end for
end while
if the open set is exhausted without reaching G **then**
  fall back to standard A*;
end if

堆中的节点比较规则定义为：

$$\begin{cases} n_1, & f(n_1) < f(n_2) \\ n_2, & f(n_1) > f(n_2) \\ n_1, & f(n_1) = f(n_2) \wedge h(n_1) < h(n_2) \\ n_2, & f(n_1) = f(n_2) \wedge h(n_1) > h(n_2) \end{cases} \tag{10}$$

该比较规则直接嵌入二叉堆的上浮（`bubbleUp`）与下沉（`bubbleDown`）操作中。从物理意义上理解，由评估函数 $f(n) = g(n) + h(n)$ 可知，当两个节点的 f 值相同时，g 值较大者 h 值较小。g 值较大表示该节点距起点更远，而 h 值较小表示其更接近目标。优先扩展此类节点使搜索偏向目标方向，减少了远离目标区域的无效探索。由于 h 值在节点扩展时已经计算，此打破平局策略不产生额外计算开销。

B. 路径后处理

JPS 搜索得到的路径由栅格中心点序列组成，存在两个主要问题：路径包含冗余的中间点，且由离散的水平和垂直段组成导致平滑性不足。本节提出一种两阶段后处理流水线来解决这些问题。

1) 安全感知路径简化: 传统的路径简化方法（如拐角裁剪）直接连接路径上的非相邻节点，跳过中间点以减少路径点数量。然而，这类方法通常仅检查连接线段是否穿越障碍物，忽略了路径点与障碍物之间的安全裕度，在障碍物密集区域可能导致路径过于贴近障碍物。为此，本节提出一种安全距离感知的路径简化算法，其核心思想是在简化过程中保持路径与障碍物之间的最小安全距离。该算法采用四步流程处理输入路径：首先从路径中识别方向发生变化的节点（即拐点）。对于路径中的每个中间节点，计算其与前后节点的连接方向。若前向连接方向与后向连接方向不同，则该节点标记为拐点。路径的起点和终点始终保留。其次从当前拐点 C_i 出发，正向扫描所有后续拐点 C_j（$j > i$），调用 IsLineFree 逐一验证线段 $C_i \to C_j$ 是否满足安全距离约束，并记录**最远的**可达拐点 j^*。该拐点成为新的锚点，其间的所有拐点被跳过。这种贪心策略在保证安全性的前提下最大化了单步简化程度。在此基础上，贪心步骤确定最远可达拐点 j^* 后，算法检查被跳过的拐点 k（$i < k < j^*$）能否通过安全线段直达更远处的拐点 j'（$j' > j^*$）。对每一对满足安全约束的 (k, j')，算法计算并比较两条路径的代价。若存在这样的中间拐点，则可能得到比纯贪心更短的简化路径。随后比较两条路径的长度：路径 A（贪心）为 $d(C_i, C_{j^*}) + d(C_{j^*}, C_{j'})$，即先到达 j^* 再继续到 j'；路径 B（经中间点）为 $d(C_i, C_k) + d(C_k, C_{j'})$，即跳过 j^* 改经 k 路由。若路径 B 更短，则保留中间拐点 C_k；否则仅保留贪心结果 C_{j^*}。这一比较确保中间点探索不会以牺牲路径效率为代价。安全性检查函数

IsLineFree$(p_1, p_2, \text{map}, d_{\text{safe}})$ 的实现基于逐栅格采样：从 p_1 到 p_2 沿线段密集采样，对每个采样点计算其与最近障碍物栅格边界的距离。只要任意采样点的距离小于安全阈值 d_{safe}，即判定该连接不安全。点到栅格边界的距离计算公式为：

$$\begin{aligned} d_x &= \max\left(0,\ |p_r - c_r| - 0.5\right) \\ d_y &= \max\left(0,\ |p_c - c_c| - 0.5\right) \end{aligned} \tag{11}$$

$$d(p, \text{cell}) = \sqrt{d_x^2 + d_y^2} \tag{12}$$

其中，$p = (p_r, p_c)$ 为连续坐标下的采样点，$c = (c_r, c_c)$ 为障碍物栅格的中心坐标，栅格占据矩形区域 $[c_r - 0.5, c_r + 0.5] \times [c_c - 0.5, c_c + 0.5]$。$\max(0, \cdot)$ 算子处理采样点在某坐标轴上已落入栅格水平或垂直范围内的情况。当 $d(p, \text{cell}) < d_{\text{safe}}$ 时，表示采样点已进入障碍物栅格的安全禁区。安全距离阈值 d_{safe} 取为机器人半径 r_{robot} 与附加安全裕度之和，从而保证路径中心线到障碍物的间距不小于机器人半径，避免碰撞。算法 2给出了路径简化算法的伪代码。安全性检查函数 IsLineFree 的伪代码如算法 3所示。搜索半径 $d_{\max}$ 设置为 $\lceil d_{\text{safe}} + 0.5 \rceil$，因为超出此距离的栅格不可能与采样点的安全距离产生交集。

2) 弧长参数化三次样条平滑: 路径简化后的路径虽已去除冗余点，但仍由直线段连接组成，在拐点处存在曲率突变，不适合全向机器人的平滑运动。本节采用三次样条插值对简化路径进行平滑处理。传统的三次样条插值直接以路径点的索引作为参数进行拟合，当路径点间距不均匀时会导致曲线形状失真。为解决这一问题，本节采用弧长参数化方法，以路径点之间的累积欧氏距离作为参数，使参数空间与实际几何空间保持一致。平滑过程分为三个步骤：第一步是稀疏段致密化：检测路径中相邻点间距超过 2 个栅格单位的线段。对于这些稀疏段，在两端点之间沿线段线性插值 $\lfloor \ell/2 \rfloor$ 个中间点。致密化的目的是为样条曲线提供足够的控制点，防止曲线在长直段处因控制点不足而产生的过冲（Runge 现象），避免曲线偏离预期折线甚至裁剪障碍物角点。第二步进行弧长参数化。设致密化后的路径点序列为 $\{P_0, P_1, \ldots, P_M\}$，其中 $P_i = (x_i, y_i)$ 为连续坐标。计算各点的累积弦长（弧长）参数：

$$s_0 = 0, \quad s_i = s_{i-1} + \|P_i - P_{i-1}\|_2, \quad i = 1, 2, \ldots, M \tag{13}$$

然后分别以 s 为自变量对 $x(s)$ 和 $y(s)$ 进行三次样条插值，得到参数化曲线 $(x(s), y(s))$。三次样条在每个子区间 $[s_i, s_{i+1}]$ 上为三次多项式，保证一阶和二阶导数连续，从而确保曲线的曲率连续性。第三步进行等弧长重采样。以插值密度 ρ 沿弧长参数 s 均匀采样，生成最终的平滑路径点序列。由于 s 与实际几何距离近似线性关系，等弧长采样保证了路径点在空间中的均匀分布，避免了传统等参数采样在曲率大处点密、曲率小处点疏的问题。算法 4给出了平滑算法的伪代码。

Algorithm 2 Safety-distance-aware path simplification

Require: grid path $P = [P_1, \ldots, P_N]$, occupancy grid, safety margin d_{safe}
Ensure: simplified path P_s ($|P_s| \le N$)

$C \leftarrow$ the corner points of P where the direction changes, plus the two endpoints;
$P_s \leftarrow [C_1]$; $i \leftarrow 1$;
while $i < |C|$ **do**
  $j^* \leftarrow$ the farthest corner reachable from C_i by a collision-free segment; *greedy forward scan*
  if $j^* = |C|$ **then**
    append $C_{|C|}$ to P_s and break;
  end if
  $(k^*, j') \leftarrow$ among all skipped corners k ($i < k < j^*$) and farther corners j ($j^* < j \le |C|$), the pair with the largest j such that the segment $C_k \to C_j$ is collision-free;
  if k^* exists and $d(C_i, C_{k^*}) + d(C_{k^*}, C_{j'}) < d(C_i, C_{j^*}) + d(C_{j^*}, C_{j'})$ **then**
    append C_{k^*} and $C_{j'}$ to P_s, and set $i \leftarrow j'$; *route through the intermediate corner*
  else
    append C_{j^*} to P_s, and set $i \leftarrow j^*$;
  end if
end while

return P_s;

3) 流水线集成: 上述后处理操作以流水线方式集成：JPS 搜索（算法 1）输出原始栅格路径 → SimplifyPath（算法 2）去除冗余点 → SmoothPath（算法 4）生成平滑的连续坐标路径。整个流水线在 SimulationManager 中一次性完成，输出的路径既满足安全性约束，又具有良好的平滑性，可直接用于机器人

Algorithm 3 IsLineFree

Require: segment endpoints p_1, p_2 (grid coordinates), occupancy grid, safety margin d_{safe}

Ensure: true if the segment keeps distance $\geq d_{\text{safe}}$ from every obstacle, false otherwise

$\ell \leftarrow \max(|p_1.r - p_2.r|,\ |p_1.c - p_2.c|)$;
$N_s \leftarrow \max(\lceil 10 \cdot \ell \rceil, 30)$; *number of dense sample points*
$d_{\max} \leftarrow \lceil d_{\text{safe}} + 0.5 \rceil$; *search radius in cells*
for $k = 0$ to N_s **do**
  $p \leftarrow p_1 + (k/N_s) \cdot (p_2 - p_1)$; *sample point in continuous coordinates*
  for each cell (r, c) within Chebyshev distance $d_{\max}$ of p **do**
    if (r, c) is occupied **then**
      $d \leftarrow$ the distance from p to the boundary of cell (r, c); *see Eqs.* (11)–(12)
      if $d < d_{\text{safe}}$ **then**

        return false;
      end if
    end if
  end for
end for

return true;

Algorithm 4 Arc-length parameterized cubic spline smoothing

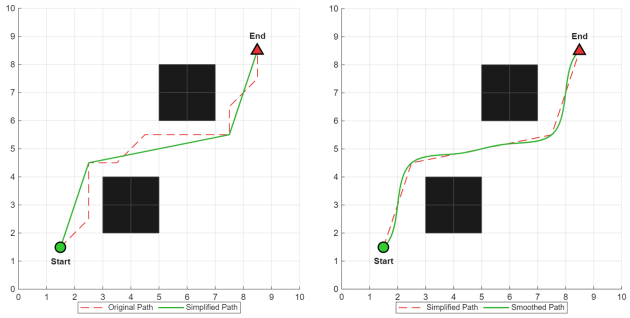
Require: grid path $P = [P_1, \ldots, P_N]$, density ρ (default 10)

Ensure: smoothed continuous path $Q = [Q_1, \ldots, Q_M]$ in $[x, y]$

convert each grid point $P_i = (r_i, c_i)$ to continuous coordinates $(x_i, y_i) = (c_i - 0.5,\ r_i - 0.5)$;
for each consecutive pair whose segment length exceeds 2 **do**
  insert $\lfloor \text{length}/2 \rfloor$ equally spaced intermediate points along the segment; *densification to prevent spline overshoot*
end for
compute the cumulative chord length $t_1 = 0$, $t_i = t_{i-1} + \|P_i - P_{i-1}\|_2$; *arc-length parameter*
remove duplicate t values to keep strict monotonicity;
fit cubic splines $x(t)$ and $y(t)$ through the densified points;
sample $x(t)$ and $y(t)$ at ρ points per segment to obtain Q;

return Q;

的运动跟踪。这种两阶段设计的合理性在于：路径简化和样条平滑分别解决了路径质量的两个不同维度。简化减少了路径点数量（降低了后续计算的复杂度），同时通过安全距离约束保证了路径的安全裕度；平滑则将离散的栅格路径转化为连续曲线，使机器人能够以平滑的运动轨迹跟踪路径。若跳过简化直接进行平滑，样条曲线需要处理大量冗余的控制点，不仅计算成本更高，还可能因控制点过于密集而产生不稳定的曲线形状。消融实验（表 VI）的结果也验证了这一设计的有效性。路径后处理前后对比如图 2所示。



(a) 原始路径与简化后路径　(b) 简化后路径与平滑后路径

图 2. 路径通过管线各阶段的对比。

C. 改进的开放旅行商蚁群优化

本框架的第二层在全局规划层产生的代价矩阵上求解开放旅行商问题。给定由固定起点 S、K 个中间目标点 $\{T_1, \ldots, T_K\}$ 和固定终点 G 组成的 N 个点，目标是找到使总路径代价最小的 K 个目标点排列。我们在蚁群优化（ACO）元启发式方法基础上引入六个集成机制：（1）虚拟节点法，将开放旅行商问题转化为标准闭环旅行商问题；（2）最大-最小蚁群系统（MMAS）信息素管理及动态更新边界；（3）蚁群系统（ACS）风格的伪随机比例转移规则，平衡探索与利用；（4）以 k 近邻

候选列表加速的可变邻域下降（VND）局部搜索；（5）S 形渐进调度策略，将搜索从探索主导逐渐转向利用主导；（6）自适应停止准则。

1) 虚拟节点法编码开放旅行商问题: 在标准闭环旅行商问题中，解为哈密顿回路，任意城市均可作为起始点。在我们的问题中，起点 S 和终点 G 是由任务规格确定的物理上不同的位置——例如无人公交的始发站和终点站。这两个点必须出现在每个候选解的固定位置（首尾），优化仅针对 K 个中间目标点的排序进行。将所有 N 个点对称排列的朴素编码会产生将起点/终点混入路径中间的无效路径；而将起点/终点固定在路径两端的边界固定编码将其排除在信息素模型和局部搜索之外，限制了算法对完整解的优化能力。为此，本文采用虚拟节点（哑节点）法将开放旅行商问题转化为标准闭环旅行商问题，该技术在开放车辆路径问题中被广泛采用 [31], [32]。引入虚拟节点 $V = N + 1$，构造 $(N + 1) \times (N + 1)$ 的扩展代价矩阵 $\mathbf{D}_{\text{ext}}$：

$$\mathbf{D}_{\text{ext}}(i,j) = \begin{cases} 0, & \{i,j\} = \{V,1\} \\ 0, & \{i,j\} = \{V,N\} \\ \text{BIG}, & (i = V \vee j = V) \wedge \\ & \{i,j\} \cap \{1,N\} = \varnothing \\ \mathbf{D}(i,j), & \text{otherwise} \end{cases} \tag{14}$$

其中 $\text{BIG} = 2(N+1) \cdot \max_{i,j} \mathbf{D}(i,j)$ 为严格大于任何可行回路代价的惩罚值，确保虚拟节点在任何近优回路中不直接连接任何中间目标点。虚拟节点与起点和终点之间的零代价边使虚拟节点对路径代价 “不可见”，而与中间目标点之间的惩罚边强制虚拟节点仅与起点和终点相邻。蚁群优化求解器在扩展代价矩阵 $\mathbf{D}_{\text{ext}}$ 上作为标准闭环旅行商问题运行：所有蚂蚁构建访问包括虚拟节点的 $N + 1$ 个节点并形成完整哈密顿回路，信息素矩阵 $\boldsymbol{\tau}$ 覆盖完整的 $(N + 1) \times (N + 1)$ 空间。求解器收敛后，通过以下步骤从最优回路中提取开放路径访问顺序：

1) 定位回路中的虚拟节点 V。
2) 移除 V 并按回路顺序读取剩余 N 个节点。
3) 调整方向使起点出现在首位、终点出现在末位。

该转换相对边界固定编码有三个优势：（a）起点和终点节点完整参与信息素模型和局部搜索，而非被排除为固定边界；（b）所有标准闭环旅行商算子统一适用，无需特殊边界处理；（c）代价评估为 $\mathbf{D}_{\text{ext}}$ 上的简单回路求和，避免了单独边界代价项的需求。启发式可见度矩阵 $\boldsymbol{\eta}$ 在扩展空间上定义。对涉及虚拟节点的点对，虚拟节点到起点或终点的零代价边赋予大启发式值 BIG，强烈吸引虚拟节点连接其所需邻居。对其他点对：

$$\eta_{ij} = \begin{cases} \dfrac{1}{\mathbf{D}_{\text{ext}}(i,j)}, & \mathbf{D}_{\text{ext}}(i,j) \in (0, +\infty) \\ 0, & \text{otherwise} \end{cases} \tag{15}$$

闭合回路 $\boldsymbol{\sigma} = (\sigma_1, \sigma_2, \ldots, \sigma_{N+1})$ 的代价为：

$$C(\boldsymbol{\sigma}) = \sum_{k=1}^{N} \mathbf{D}_{\text{ext}}(\sigma_k, \sigma_{k+1}) + \mathbf{D}_{\text{ext}}(\sigma_{N+1}, \sigma_1) \tag{16}$$

2) 具有动态边界的最大-最小蚁群系统信息素更新: 在基本蚁群系统中，所有蚂蚁在其完整路径上沉积信息素，当少数蚂蚁偶然找到中等质量的路径时可能导致搜索过早集中于次优解。反之，无边界的过度蒸发可能使信息素值在除主导边之外的所有边上趋近于零，将种群困在局部最优。最大-最小蚁群系统（MMAS）[23] 通过以下方式解决这两个问题：（a）将信息素限制在显式区间 $[\tau_{\min}, \tau_{\max}]$ 内，防止支配和灭绝；（b）聚焦于最优性能解的沉积。据此，每次迭代时所有信息素值首先进行蒸发：

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} \quad \forall i,j \in \{1, \ldots, N+1\} \tag{17}$$

其中 ρ 为蒸发率。蒸发后，在两个解层次的边上沉积信息素：蒸发之后，当代最优蚂蚁在其回路的所有边上沉积信息素：

$$\tau_{ij} \leftarrow \tau_{ij} + \frac{Q}{C_{\text{iter}}} \quad \forall (i,j) \in \text{edges}(\boldsymbol{\sigma}_{\text{iter-best}}) \tag{18}$$

同时，搜索开始以来找到的最优回路也获得沉积，以强化全局主导解：

$$\tau_{ij} \leftarrow \tau_{ij} + \frac{Q}{C_{\text{global}}} \quad \forall (i,j) \in \text{edges}(\boldsymbol{\sigma}_{\text{global-best}}) \tag{19}$$

其中 Q 为沉积常数，C 为回路代价。沉积量与回路代价成反比，因此更短（更优）的回路沉积更强的信息素。两个沉积级别同时应用于对称对 (j,i)，保持无向信息素模型。沉积后，所有信息素值被限幅到 $[\tau_{\min}, \tau_{\max}]$ 区间，其边界根据当前全局最优动态更新：

$$\tau_{\max} = \frac{1}{\rho \cdot C_{\text{global}}} \tag{20}$$

$$\tau_{\min} = \frac{\tau_{\max}}{N+1} = \frac{1}{\rho \cdot (N+1) \cdot C_{\text{global}}} \tag{21}$$

并设有硬下限 $\tau_{\min}$ 以防止低频边上信息素完全灭绝。边界的动态更新至关重要：随着搜索发现更优回路（C_{global}

减小），$\tau_{\max}$ 增大，允许算法对其当前最优解表达更强的置信。反之，比率 $\tau_{\min}/\tau_{\max} = 1/(N+1)$ 维持最小与最大信息素之间的恒定相对间隔，无论绝对尺度如何，都为所有边保持基线探索概率。初始信息素矩阵设为均匀值 τ_0，表示搜索经验积累前的中性先验。

3) 蚁群系统风格的伪随机转移规则： 在标准蚁群系统中，每只蚂蚁通过纯概率轮盘选择从未访问的候选集中选择下一节点。此方法探索性强，在距离启发式提供可靠引导的结构化问题上收敛缓慢。蚁群系统（ACS）[22] 通过伪随机比例规则解决此问题：用可控概率使蚂蚁贪心地选择最优候选；否则通过标准概率机制探索。这平衡了收敛需求（利用）与避免局部最优的需求（探索）。具体实现时，当蚂蚁在当前节点 u 从未访问候选集 $\mathcal{U}$ 中选择下一节点时，首先计算每个候选的得分：

$$\mathrm{score}(v) = (\tau_{uv})^{\alpha} \cdot (\eta_{uv})^{\beta} \quad \forall v \in \mathcal{U} \tag{22}$$

其中 α 和 β 分别为信息素和启发式权重指数。转移决策遵循：

$$v_{\mathrm{next}} = \begin{cases} \arg\max_{v \in \mathcal{U}} \ \mathrm{score}(v), & \text{概率} q_0 \\ \mathrm{roulette}(\mathcal{U}, \mathrm{score}), & \text{概率} 1 - q_0 \end{cases} \tag{23}$$

其中 q_0 为利用概率。在利用情形中，蚂蚁确定性地选择得分最高的候选——等价于贪心最优步。在探索情形中，蚂蚁执行轮盘选择，每个候选 v 以 $\mathrm{score}(v)/\sum_{u\in\mathcal{U}}\mathrm{score}(u)$ 的概率被选中。若所有得分为零即所有剩余节点不可达，则随机选择一个未访问候选作为回退。可以看出，q_0 越大，越多的构建步采用贪心选择以加速收敛；q_0 越小，则随机探索的比重越高。障碍约束代价矩阵中的点对距离通常具有空间局部性，因而启发式 $\boldsymbol{\eta}$ 能够提供较可靠的引导，这使得中等偏上的贪心概率在保证多样性的前提下仍有利于收敛。

4) 具有候选列表加速的可变邻域下降局部搜索： 蚂蚁所构建的路径虽受信息素与启发式信息引导，但仍未必达到局部最优。一个自然的改进思路是：在每一代对当前最优解施加局部搜索，通过系统考察邻域排列来进一步精化解的质量。可变邻域下降（Variable Neighborhood Descent, VND）[24] 正是这样一类方法，它依次调用多种邻域结构，每种结构对应一类路径变换，并在当前结构无法继续改进时切换到下一种，直至所有邻域均无法产生更优解。已有研究表明，将 MMAS 或 ACS 与 VND 混合在车辆路径问题中效果显著 [25]。不过，标准 VND 在每个算子上需考察 $O(K^2)$ 个邻域，当目标点规模增大时计算开销会迅速上升，成为主要瓶颈。进一步观察可以发现，这类考察中大量尝试并不奏效：相距较远的节点对参与 2-opt 交换时，往往很难带来回路代价的改善。

针对上述问题，本文在包含虚拟节点在内的 $N+1$ 个节点所构成的闭合回路上实施 VND，并按固定顺序交替使用三种邻域算子。各算子的作用及配套的候选列表剪枝条件如下：

1) **2-opt**：将路径中一段连续子序列整体反转，即用边 (u, v) 与 (u^+, v^+) 替换原边 (u, u^+) 与 (v, v^+)，从而消除欧氏型代价下常见的路径交叉。评估该翻转前，仅当新边 (u, v) 与 (u^+, v^+) 中至少有一对属于候选近邻时，才计算代价变化。
2) **重定位**：从当前位置取出某一节点，将其插入回路的其他位置，并相应平移中间节点，用于纠正节点被错误放置到不同路径区段的情形。评估该插入前，仅当被移动节点 v 与其新邻居之一（p 或 p^+）互为候选近邻时，才继续考察。
3) **交换**：在保持其余节点不变的前提下对调两个节点的位置，适用于两节点恰被安排到对方更优位置的情况。评估该对调前，仅当待交换的两个节点 a 与 b 互为候选近邻时，才考虑执行。

上述算子在 VND 循环中采用“首次改进”（first-improvement）策略：先反复进行 2-opt，直至无法再降低代价；若仍无改进则转为重定位，最后尝试交换。只要任一算子产生改进，便重新从 2-opt 开始下一轮循环；当三种算子均无法进一步改进时，局部搜索终止。

在计算效率方面，本文引入预计算的候选列表，以过滤上述各类评估中明显缺乏前景的情形，这一思路在旅行商问题的局部搜索中已有广泛应用 [26]。具体而言，对扩展问题中的每一个节点（共 $N+1$ 个），根据扩展代价矩阵 $\mathbf{D}_{\mathrm{ext}}$ 预先筛选其 k 个最近邻，并以布尔矩阵 $\mathbf{C}$ 加以存储，其中 $\mathbf{C}(i, j) = \mathrm{true}$ 表示节点 j 属于节点 i 的候选近邻；上文各算子中所提到的“候选近邻”均指该列表中的节点。需要说明的是，上述剪枝在设计上偏于保守：只有当移动所涉及的新边均不在候选近邻集合中时，才跳过评估。这种情形通常意味着该移动难以改善回路，但并不能视为绝对保证。候选列表规模 k 宜取得较小，既可过滤掉大量无效计算，又保留最有可能带来改进的局部结构；当扩展代价矩阵呈现较强的空间聚集特性时，改进边交换几乎只可能发生在邻近节点之间，因此该设置尤为有效。

从复杂度来看，若不采用剪枝，每个 VND 算子需考察 $O((N+1)^2)$ 个候选移动；引入候选列表后，每个节点仅需参与约 $O(k)$ 次邻域检查，单算子复杂度降为 $O((N+1)\cdot k)$。当 k 取个位数量级时，相对全邻域考察约可降低一个数量级的计算量；随着 N 增大，加速效果会更加明显。

算法 5给出了候选列表加速 VND 的完整伪代码。需要补充的是，上述三个邻域算子均按首次改进原则运行，即一旦发现能够降低回路代价的移动便立即接受并返回；由于问题定义在闭合回路上，当交换操作跨越回路的首尾相邻节点时，还需额外考虑环绕邻接关系，而不能简单沿用线性路径下的邻接判断。VND 邻域搜索的完整流程如图 3所示。如前所述，三个邻域算子均按首次改进原则运行，即一旦发现能够降低回路代价的移动便立即接受并返回；由于问题定义在闭合回路上，当交换操作跨越回路的首尾相邻节点时，还需额外考虑环绕邻接关系，而不能简单沿用线性路径下的邻接判断。VND 邻域搜索的完整流程如图 3所示。

5) S 形渐进局部搜索调度: 即使引入了候选列表加速，VND 局部搜索仍需要可观的计算时间。若在每一代对所有蚂蚁都做局部搜索，早期迭代的资源往往会被浪费：此时种群尚未辨明搜索空间中较有希望的区域，精化低质量路径的收益有限。相反，到了后期，当信息素已集中于高质量区域时，密集局部搜索对于进一步打磨解却十分关键。可见，若始终对固定比例的蚂蚁实施局部搜索并不理想——比例偏低会减慢收敛，偏高则在尚未成熟的解上耗费算力。为此，本文在迭代过程中按 S 形（sigmoid）函数动态调整接受 VND 局部搜索的蚂蚁比例：

$$\mathrm{optRatio}(t) = y_{\min} + (y_{\max} - y_{\min}) \cdot \frac{t^a}{t^a + (1-t)^a} \quad (24)$$

式中，$t = (\mathrm{iter} - x_L)/(x_R - x_L)$ 为当前迭代在转换区间 $[x_L, x_R]$ 内的归一化位置；$y_{\min}$ 与 $y_{\max}$ 分别表示搜索初期与后期接受局部搜索的蚂蚁比例上下限，a 用于控制 S 形曲线的陡峭程度。当 t 位于 x_L 之前时，optRatio 取 $y_{\min}$；超过 x_R 之后则保持为 $y_{\max}$；在区间内部，比例沿 S 形曲线平滑过渡。

这一调度方式使搜索过程自然形成三个阶段。早期 $\mathrm{optRatio} \approx y_{\min}$，种群主要依赖路径构建阶段的随机性来广泛探索，而不必承担局部搜索开销；在过渡阶段，接受优化的蚂蚁逐步增多，使信息素能够吸收由局部搜索得到的改进解；后期 optRatio 稳定在 $y_{\max}$，在继续由部分蚂蚁维持构建多样性的同时，以足够多的算力精化当前优质解。转换区间之所以取得较短的 $[0, 30]$，是为了让比例尽早达到上限：实验表明，基本信息素结构往往在约 30 代内形成，此后加密局部搜索即可获得明显收益。S 形 $\mathrm{optRatio}(t)$ 曲线如图 4所示。在确定本轮接受局部搜索的蚂蚁数目 $n_{\mathrm{opt}} = \lceil n_{\mathrm{ants}} \cdot \mathrm{optRatio} \rceil$ 之后，还需从中挑选具体个体。本文采用精英与随机相结

Algorithm 5 Candidate-list-accelerated variable neighborhood descent

Require: closed tour $\boldsymbol{\sigma} = [\sigma_1, \ldots, \sigma_{N+1}]$, extended cost matrix $\mathbf{D}_{\mathrm{ext}}$, candidate boolean matrix $\mathbf{C}$
Ensure: improved tour $\boldsymbol{\sigma}$ and its cost

cost ← the cost of tour $\boldsymbol{\sigma}$ under $\mathbf{D}_{\mathrm{ext}}$;
repeat
  improved ← false;
  $[\boldsymbol{\sigma}, \mathrm{cost}, \mathrm{ok}]$ ← apply the 2-opt operator with candidate-list pruning (first-improvement);
  improved ← improved or ok;
  $[\boldsymbol{\sigma}, \mathrm{cost}, \mathrm{ok}]$ ← apply the relocate operator with candidate-list pruning;
  improved ← improved or ok;
  $[\boldsymbol{\sigma}, \mathrm{cost}, \mathrm{ok}]$ ← apply the swap operator with candidate-list pruning;
  improved ← improved or ok;
until improved = false
cost ← the exact cost of the final tour $\boldsymbol{\sigma}$;

return $\boldsymbol{\sigma}$, cost;

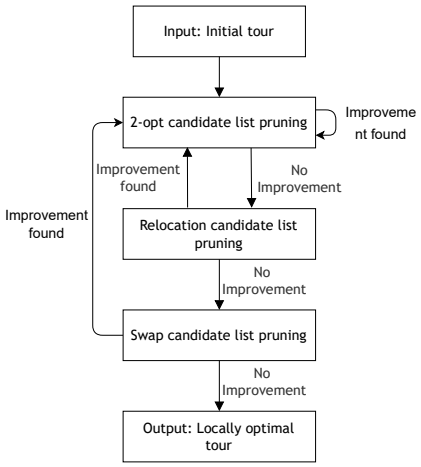


图 3. VND 邻域搜索序列流程图。

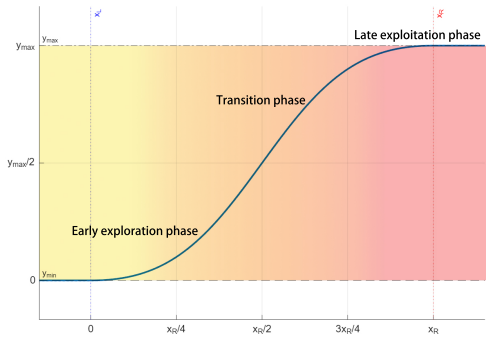


图 4. S 形 optRatio(t) 关于迭代次数的曲线图。

合的方式：

$$n_{\text{elite}} = \lceil n_{\text{opt}} \cdot r_{\text{elite}} \rceil \tag{25}$$

$$n_{\text{random}} = n_{\text{opt}} - n_{\text{elite}} \tag{26}$$

其中 r_{elite} 为精英占比参数，即优先把较多的局部搜索预算用于当前排名靠前的蚂蚁，以强化对优质解的精细打磨；其余 n_{random} 只蚂蚁则从非精英个体中随机抽取并同样接受优化，从而有机会发现精英之外的其他有潜力区域，在利用与探索之间保持平衡。

6) 自适应停止准则： 算法将最大迭代次数 n_{iter} 作为迭代预算上限，以免在困难实例上因预算不足而无法收敛。但在较易实例中，种群往往远在此之前就已收敛，继续迭代只会增加计算开销，而难以带来解质量的提升。反过来，若过早停止，又可能错过更优解。因此，停止准则需要在“已充分收敛”与“暂时尚未改进”之间加以区分，既识别真正的收敛，也避免因短暂停滞而误停。

具体而言，算法同时监测两个互补的指标。

第一个指标是种群同质性。当蚁群收敛到搜索空间中某个较窄的区域时，不同蚂蚁给出的回路代价会彼此接近，方差随之变小。本文用每代蚂蚁回路代价的变异系数（coefficient of variation, CV）对此加以刻画：

$$\text{CV} = \frac{\sigma_{\text{costs}}}{\mu_{\text{costs}}} \tag{27}$$

其中 σ_{costs} 与 μ_{costs} 分别为当代全部蚂蚁回路代价的标准差与均值。当 CV 低于预设阈值 ε_{CV} 时，可认为种群已趋于同质，即各回路质量大体相当，继续搜索发现新解的可能性已经很低。

第二个指标是最优解是否陷入停滞。即使种群内部仍有一定多样性，当前全局最优回路也可能长时间停留在某个局部最优附近。为此，算法记录全局最优代价连续未获改进的迭代次数，一旦达到预设上限 $\text{stagn}_{\max}$，便终止搜索。

为减少误停风险，上述两个判据仅在完成最少迭代次数 minIter 之后才启用。这是因为在搜索初期，代价方差通常较大、全局最优仍在快速改善，此时若立即监测收敛，容易把正常波动误判为已收敛；经过一段最低限度的搜索后，信息素结构已相对成形，此时给出的停止判断才更为可靠。

7) 集成总结： 综合上述设计，MMAS-VND-CL 算法把六个机制组织进同一条迭代搜索流程：扩展代价矩阵 $\mathbf{D}_{\text{ext}}$ 定义在含虚拟节点的 $N+1$ 个节点上，信息素模型与局部搜索均在该空间中进行。单次迭代的主要步骤如下：

1) 由 S 形曲线（式 (24)）确定本轮接受局部搜索的蚂蚁比例；
2) 全部蚂蚁按照 ACS 风格的伪随机转移规则（式 (22)–(23)）各自构造一条覆盖 $N+1$ 个节点的哈密顿回路；
3) 依据精英—随机混合策略（式 (25)–(26)）选出部分蚂蚁，对其回路执行带候选列表加速的 VND 局部搜索（算法 5）；
4) 更新当前全局最优回路，并依据自适应停止准则（式 (27)）判断是否终止；
5) 若继续搜索，则先按式 (17)蒸发信息素，再在本轮最优回路与全局最优回路上沉积（式 (18)–(19)），最后按 MMAS 规则对信息素取值进行动态限幅（式 (20)–(21)）。

搜索在满足任一停止条件时结束，即种群代价的变异系数低于 ε_{CV}，或全局最优回路连续 $\text{stagn}_{\max}$ 代没有改进；为避免误停，这些条件在 minIter 之前不参与判定。终止后，从最优回路中去掉虚拟节点 V，再把剩余节点调整为起点在前、终点在后的顺序，便得到最终的开放路径访问序列。MMAS-VND-CL 算法的总体流程如图 5所示。

D. 协同优化框架

前述各节分别介绍了两个算法层：负责点对点寻径的改进 A* 规划器及其后处理（第 IV-A–IV-B 节），以及面向开放旅行商问题的 MMAS-VND-CL 求解器（第 IV-C 节）。本节说明如何将二者衔接为一个完整的多目标点遍历框架。

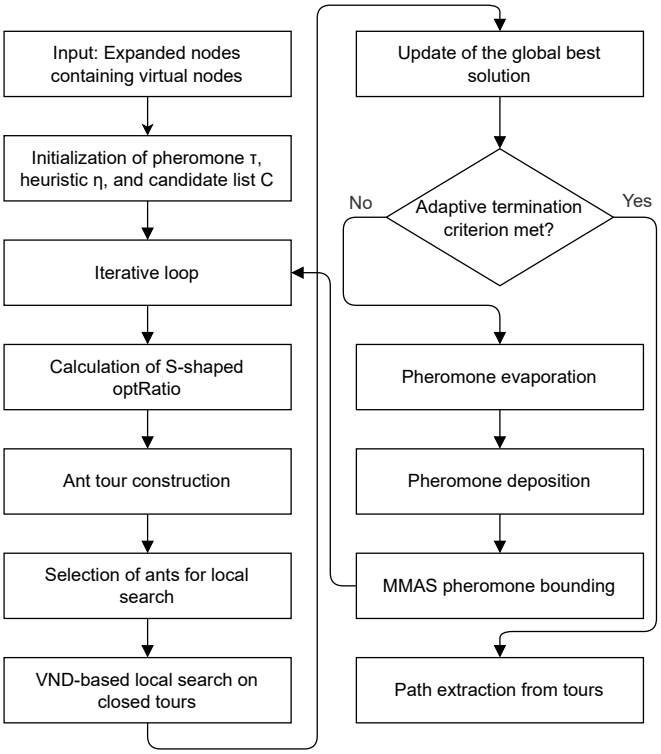


图 5. 改进蚁群优化总体流程图。

1) 两层架构: 该框架在结构上分为两层，从而把几何层面的避障寻径与组合层面的访问顺序优化清晰地分开。

第一层完成代价矩阵的构建。给定 N 个点（含一个起点、K 个中间目标点和一个终点），对每一对点计算最短无碰撞路径，由此得到成对代价矩阵 $\mathbf{D} \in \mathbb{R}^{N\times N}$。矩阵元素 $\mathbf{D}(i,j)$ 定义为从点 i 到点 j 的最短无碰撞路径长度，对角元素取零。需要强调的是，$\mathbf{D}$ 并非欧氏距离矩阵：它的每一个元素都对应真实的障碍约束最短路径长度。当障碍物截断两点之间的直线通道时，该长度可能明显大于两点间的直线距离。

为使代价更贴近机器人实际行程，路径长度并不直接采用栅格上的步进累加值。具体而言，先对全局规划得到的栅格路径作安全感知的简化，再经弧长参数化三次样条平滑得到连续轨迹，最后以该平滑轨迹上相邻采样点之间欧氏距离的累加和作为 $\mathbf{D}(i,j)$。这样既削弱了栅格离散化的阶梯效应，又更加符合实际的运动轨迹。构建完整代价矩阵共需调用 $O(N^2)$ 次全局规划器；工程上还可选用以起止坐标对为键的路径缓存，保存已经算过的点对结果，从而避免在多次旅行商求解中重复进行 A* 搜索。

第二层完成访问顺序的优化。将代价矩阵 $\mathbf{D}$ 交给 MMAS-VND-CL 求解器即可。该层只在数值矩阵上操作，并不感知地图、障碍物或路径几何；全部约束信息都已压缩在 $\mathbf{D}$ 之中。求解器返回最优访问顺序 $\boldsymbol{\pi}^* = (1, \pi_2, \ldots, \pi_{K+1}, N)$ 及对应总代价 $C(\boldsymbol{\pi}^*)$。

2) 管线集成: 完整管线可分为如下四个阶段。

点集组装。将 $N = K + 2$ 个点写入统一的坐标数组 $\mathbf{P} = [p_{\text{start}};\ \mathbf{T}_1; \ldots; \mathbf{T}_K;\ p_{\text{goal}}]$，并约定索引 1 为起点、索引 N 为终点。

代价矩阵构建。对每一对有序点 (i,j)（$i \neq j$），由全局规划器求出从 $\mathbf{P}(i,:)$ 到 $\mathbf{P}(j,:)$ 的最短无碰撞路径，写入路径缓存；随后对该路径作安全简化与样条平滑，并将平滑后连续轨迹的欧氏弧长记入 $\mathbf{D}(i,j)$，使其更符合实际运动轨迹。若某对点之间不存在可行路径，则相应元素记为 $\mathbf{D}(i,j) = \infty$。一旦有目标点从起点不可达，流程立即中止并报告失败。

旅行商求解。将 $\mathbf{D}$ 与点数 N 交给 MMAS-VND-CL，得到最优访问顺序 $\text{bestOrder} = (1, \ldots, N)$ 及其总代价。当 $K = 0$（无中间目标）时，问题退化为平凡情形：顺序为 $[1, N]$，代价即 $\mathbf{D}(1, N)$。

输出组装。按 bestOrder 从坐标数组中取出有序点列 $\mathbf{P}(\text{bestOrder}, :)$，并在路径缓存中查回相邻点对之间的各段路径，拼成最终的无障碍参考路径集。

3) 设计原理: 将两层分开设计，主要有三方面优点。

首先，规划器与旅行商求解器可以独立开发、测试与改进。第一层可换成任意栅格规划器（A*、Dijkstra、RRT 等），第二层也可换成任意排列优化算法（蚁群优化、遗传算法、模拟退火等）。这种模块化设计在第 V-C–V-E 节的实验中得到检验：对比某一层的替代实现时，另一层保持不变。

其次，代价矩阵 $\mathbf{D}$ 充当了紧凑的接口。无论环境复杂程度如何，$N \times N$ 矩阵都完整记录了各点对之间的障碍约束距离关系。旅行商求解器因此无需接触栅格地图，却仍具备避障意义上的可行性保证。当然，这种压缩也有信息损失：求解器无法判断两段路径是否共用同一条狭窄走廊。但就“最小化总代价”这一目标而言，该表示是充分的。

再次，以平滑后连续轨迹的欧氏弧长作为点对代价，提高了组合优化所依据的距离度量的准确性。若直接对原始栅格路径按正交步、对角步累计长度，阶梯状折线会系统性地高估连续行程；经安全简化与样条平滑后，代价矩阵更接近机器人实际跟踪的路径长度，从而

有利于旅行商求解器更合理地比较不同访问顺序。

4) 完整系统流程: 端到端运行时，系统依次完成以下步骤：

1) **构建代价矩阵**：对全部 $O(N^2)$ 个点对调用改进 A* 规划器，并对所得路径进行安全简化后计价；
2) **求解开放旅行商问题**：调用 MMAS-VND-CL 确定最优访问顺序；
3) **回取各段路径**：依据访问顺序从路径缓存中拼接完整轨迹；
4) **后处理**：对每段路径实施路径简化与三次样条平滑（第 IV-B 节），输出可供机器人跟踪的平滑无障碍连续轨迹。

系统整体架构如图 6所示。

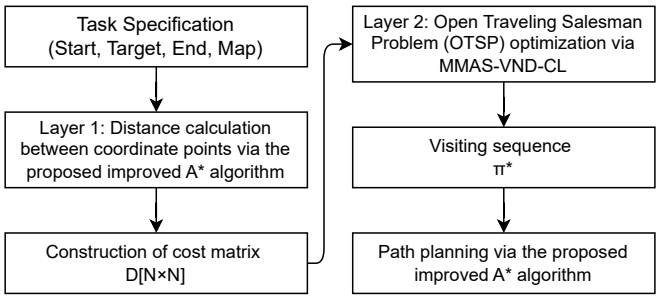


图 6. 系统架构总图

V. 实验与分析

在给出方法设计之后，本节通过仿真对所提框架的有效性进行检验。实验从四个相互衔接的层面展开：（1）比较基于 JPS 的改进 A* 规划器与常见基线在全局寻径上的效率与路径质量；（2）考察 MMAS-VND-CL 对关键超参数的敏感程度；（3）通过消融实验分离各组成机制的贡献；（4）将不同旅行商求解器与改进 A* 规划器组合，比较完整的多目标点遍历性能。所有对比均在统一参数策略下进行多次独立重复，以兼顾解质量与统计稳定性。

A. 实验设置

所有实验均在配备 Intel Core Ultra 5 125H 处理器与 32 GB 内存的计算机上完成，软件环境为 MATLAB R2025b。

为覆盖不同障碍形态与问题规模，本文构建了两类实验场景（图 7）：一类是含障碍的栅格地图（Map 1、Map 2 与 Map 3），用于检验避障寻径与多目标点遍历；另一类是标准 TSPLIB 算例 kroB100，用于单独检验大规模组合排序能力。其中 Map 1 与 Map 2 同时给出多目标遍历所需的中间目标点（图中蓝色编号点）；在全局路径规划对比实验（第 V-B 节）中，仅使用各图的起点与终点进行点对点规划，不涉及中间目标点的访问顺序优化。

具体环境特征如下：

1) Map 1：50×50 栅格环境。障碍以较长的横纵墙段为主，彼此错落布置，形成若干由走廊连通的“房间”状空腔，整体近似街区或室内隔断布局。自由空间连通性尚可，但直线通道会被墙体多次截断，适合考察规划器在中等障碍密度下的绕行与搜索效率。绿色圆点为起点（左下），红色三角为终点（右上）；蓝色编号点为多目标遍历实验中的中间目标点。
2) Map 2：80×80 栅格环境。障碍呈不规则碎块状高密度散布，自由空间被切割成大量细碎、曲折的通道，局部通行高度依赖短距离绕障。相较 Map 1，其搜索分支更多、路径阶梯效应更明显，可用于检验算法在杂乱环境中的鲁棒性。同样包含起点、终点与编号中间目标点。
3) Map 3：80×80 迷宫环境。墙段更长且大体正交布置，构成典型的迷宫式走廊，通道狭长、转折频繁，但不设置中间目标点，仅用于全局路径规划对比，以突出不同寻径算法在结构化迷宫中的表现差异。
4) kroB100：TSPLIB 标准 100 城市算例。城市以平面坐标给出，不设栅格障碍，重点在于大规模目标点的访问顺序优化。本文按开放旅行商设定，选取互不重合的固定起点与终点，中间城市全部作为待访问目标，用于检验 MMAS-VND-CL 在约百点规模上的组合搜索能力。

关于重复次数与统计口径：全局路径规划对比中每个算法在每张地图上运行 10 次并取时间平均值；旅行商相关实验中每种配置运行 40 次，报告最优代价值、平均收敛代价及标准差、运行时间、成功率统计量。

参数设置方面，对于本文提出的 MMAS-VND-CL 算法的主要参数，全文使用表 I中的默认参数值。

评估指标包括：路径长度、计算时间、扩展节点数（仅限全局规划）、最优代价、平均收敛代价、成功率（计算达到最优代价的百分比）和标准差。

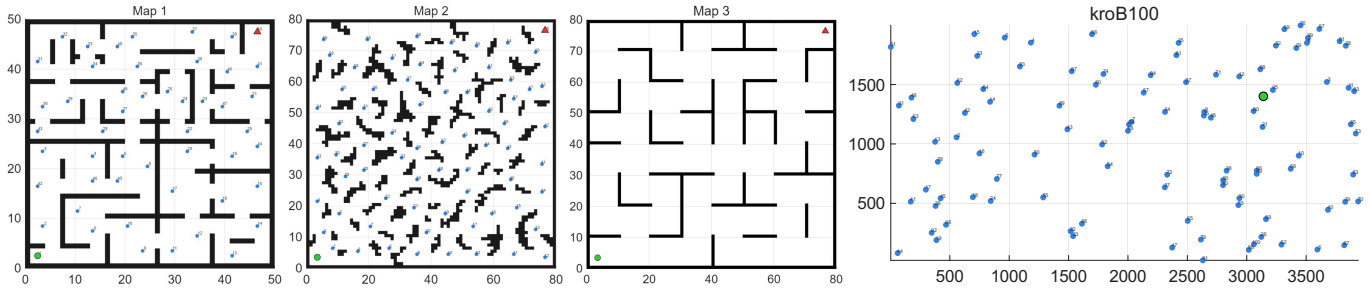


图 7. 实验环境示意图。Map 1 与 Map 2 中，绿色圆点为起点，红色三角为终点，蓝色编号点为多目标遍历的中间目标点；Map 3 仅给出起终点；kroB100 为 100 城市平面分布，绿色圆点表示开放路径的端点。

表 I
所提算法的默认参数设置

Module	Parameter	Symbol	Value
Original params	Number of ants	n_{ants}	40
	Pheromone weight	α	1
	Heuristic weight	β	2.0
	Evaporation rate	ρ	0.25
	Deposit constant	Q	225
	Initial pheromone	τ_0	0.1
ACS	Greedy probability	q_0	0.45
VND	Local search ratio	$y_{\max}$	0.4
	Local search transition interval	$[x_L, x_R]$	[0, 30]
	Elite ratio	r_{elite}	0.7
	Candidate list size	k	9
	Local search ratio lower bound	$y_{\min}$	0.0
	S-curve steepness	a	2.0
Adaptive stopping	Homogeneity threshold	ε_{CV}	10^{-3}
	Stagnation limit	$\text{stagn}_{\max}$	70
	Minimum iterations	minIter	30
	Maximum iterations	n_{iter}	800

表 II
全局路径规划性能对比

算法	地图	路径长度	时间 (ms)	扩展节点数
本文改进 A*	Map 1	72.93	2.45	70
	Map 2	108.17	11.85	415
	Map 3	113.33	4.31	48
A*	Map 1	76.53	6.14	811
	Map 2	111.44	29.19	1979
	Map 3	119.64	17.88	2796
Dijkstra	Map 1	76.53	24.26	1997
	Map 2	111.44	202.29	5217
	Map 3	119.64	204.46	5664
RRT	Map 1	100.80	7.42	—
	Map 2	135.40	19.33	—
	Map 3	144.07	3.69	—

B. 全局路径规划对比

本组实验对比所提基于 JPS 的改进 A* 规划器与传统 A*、Dijkstra 及 RRT 三种基线，从路径长度、计算时间与扩展节点数三个维度检验寻径质量与搜索效率。每个规划器均在 Map 1、Map 2 与 Map 3 上独立运行 10 次并取时间平均，结果汇总于表 II，路径可视化与长度、时间对比分别见图 8–10。总体而言，所提方法在三张地图上同时取得最短路径、更少的节点扩展与更短耗时；采样类方法可能在个别场景中以牺牲路径质量换取极短耗时，而碎块障碍会削弱 JPS 相对传统 A* 的剪枝幅度。下面结合指标、条件与算法机制展开说明。

首先看路径长度。表 II显示，所提方法在 Map 1–Map 3 上分别达到 72.93、108.17 与 113.33，较传统 A* 与 Dijkstra 依次缩短约 4.7%、2.9% 与 5.3%；图 9的折线趋势与表格一致，表明该优势并非单点偶然。需要说明的是，传统 A* 与 Dijkstra 在一致启发下给出的是栅格度量下的最优折线，二者长度相同正说明它们都触及了离散最优；所提方法更短，主要并非“搜到另一条更短的栅格路径”，而是后处理把阶梯状折线变成了更紧凑的连续轨迹——安全感知简化合并冗余航点、削减绕角，样条平滑进一步消除锯齿，从而更接近物理上的连续最短行程。换言之，长度优势与第 IV-B 节的简化—平滑机制直接对应。

其次看搜索效率。扩展节点数上，所提方法在三张图上均远低于两个图搜索基线：相对 A* 分别减少约 91.4%、79.0% 与 98.3%，相对 Dijkstra 则均减少约 92% 以上。节点扩展的压缩直接反映在耗时上：Map 3 为 4.31 ms，约为 Dijkstra 的 1/47；Map 1 为 2.45 ms；Map 2 为 11.85 ms，仍约为 Dijkstra 的 17 倍加速。这一现象可由 JPS 的对称性剪枝解释：搜索只需在被迫

邻居与方向变化处生成跳点，中间同向栅格不再逐一入堆，搜索树因而被大幅压缩；在同一地图上 Dijkstra 向各方向均匀扩展、传统 A* 虽有启发引导但仍在逐格松弛，故扩展数呈数量级差距。同时也应客观看到，Map 2 上相对 A* 的降幅（约 79%）明显小于 Map 1 与 Map 3（约 91% 与 98%）：障碍碎块使被迫邻居更频繁，跳点数量上升，剪枝收益被部分抵消，但整体上仍保持对逐格展开方法的数量级优势。

再与采样类方法比较。RRT 在 Map 3 上以 3.69 ms 成为耗时最短者，这一结果需要如实指出：当迷宫通道规则、起终点近乎直线连通时，随机树可能较快找到一条可行解。但代价是路径质量显著下降——RRT 长度在三张图上均最长，较所提方法高约 25%–38%，且多次运行表现出较大的随机波动，与采样一致性较差的典型特征相符。因而在“既要短路径、又要稳定可重复”的评价口径下，RRT 并不构成对所提方法的实质威胁；若任务仅要求快速得到任意可行解，则采样法仍有其适用空间。

综上，三个指标可以相互印证：长度优势来自简化—平滑对连续行程的逼近，扩展与耗时优势来自 JPS 对搜索树的剪枝，二者分别对应后处理管线与图搜索内核的不同机制。Map 2 上剪枝幅度相对收窄提示，在高度杂乱的栅格中被迫邻居会削弱 JPS 的对称性收益，但扩展数与耗时仍全面优于逐格展开的图搜索。后续工作可考虑在高杂乱地图上自适应调整剪枝或跳点生成策略，以进一步压低此类场景的搜索开销。

C. 参数敏感性分析

本组实验考察 MMAS-VND-CL 对五个关键超参数的敏感性，即蚂蚁数量 n_{ants}、贪心选择概率 q_0、后期局部搜索比例 $y_{\max}$、精英比率 r_{elite} 与候选列表大小 k。采用单因素变动方式：每次只将一个参数在五个水平上取值（表 III），其余参数保持默认，默认的参数水平为 3/3/3/4/2，共 25 组配置；每组在 Map 1、Map 2 与 kroB100 上各独立运行 40 次。评价不只看最优代价，还同时观察平均代价/标准差、成功率与平均收敛时间，完整结果见表 IV，代价—时间趋势见图 11，默认参数下的路径形态见图 12。完整敏感性结果见表 IV。参数水平依次为 $n_{\text{ants}}/q_0/y_{\max}/r_{\text{elite}}/k$。

对于 Map 1、Map 2 和 kroB100 三个地图，他们的最优代价分别为 351.44、620.15 和 22141。从最优代价看，表 IV中几乎所有配置都能触及各图的同一最优值，说明 MMAS-VND-CL 在合理参数范围内具备较强的“找到优解”能力；真正拉开差距的是成功率、平均代价与耗时。结合图 11可以得到的主要结论是：q_0 与 $y_{\max}$ 最为敏感，n_{ants} 居中，r_{elite} 与 k 相对温和；默认水平（3/3/3/4/2）在质量、稳定性与时间之间较为均衡，因此被后续实验固定采用。下面按参数分别说明。

表 III
敏感性分析中使用的参数水平

参数	参数水平				
	水平 1	水平 2	水平 3	水平 4	水平 5
n_{ants}	10	20	40	60	90
q_0	0.10	0.30	0.45	0.60	0.90
$y_{\max}$	0.10	0.20	0.40	0.60	0.90
r_{elite}	0.1	0.3	0.5	0.7	0.9
k	3	9	15	30	60

蚂蚁数量 n_{ants} 影响搜索广度与每代开销。当 n_{ants} 从 10 增至 40 时，kroB100 成功率由 92.5% 升至 97.5%，平均代价由 22145.03 降至 22142.45、标准差由 14.66 收窄到 9.17，而 Map 2 成功率也从 80% 升到 100%。这与机制相符：蚁群依赖多条并行回路同时采样，种群过小时信息素更新易被少数个体带偏，方差增大；种群增大则每代可沉淀更可靠的信息素结构。但继续增至 60、90 只蚂蚁时，成功率仅维持在 97.5%–100%，耗时却由约 0.35 s 升至约 0.48–0.63 s，边际收益递减。因此 $n_{\text{ants}} = 40$ 更像“够用且不贵”的折中，而非越大越好。

贪心概率 q_0 是敏感度最高的参数，对应伪随机转移中“利用”与“探索”的配比。q_0 由 0.45 降至 0.10 或 0.30 时，kroB100 仍可保持 100% 成功率，但耗时增至约 0.48–0.55 s，说明过度随机会拖慢收敛；反之，当 q_0 升至 0.90 时，此时接近为纯贪心的情况，kroB100 成功率骤降至 50%、标准差膨胀到 42.06，Map 2 成功率也仅 62.5%，平均代价升至 620.75。这一现象正对应式 (23)的作用机理：q_0 过高时，蚁群几乎总是锁定当前得分最高的候选，信息素迅速坍缩到少数边上，早熟收敛使多次运行落入不同的次优盆地，故最优值尚可触及而均值与方差恶化。默认 $q_0 = 0.45$ 使大约一半步长保留轮盘探索，恰可维持多样性又不至于盲目游走。

后期局部搜索比例 $y_{\max}$ 直接决定 VND 精化的力度。当 $y_{\max}$ 降至 0.10 时，Map 2 成功率仅 65%、kroB100 为 80%，平均代价分别升至 620.71 与 22157.05，标准差明显变大；这说明缺少后期加密局部搜索时，构建阶段得到的解难以充分打磨。当 $y_{\max}$ 升至 0.50 或 0.90

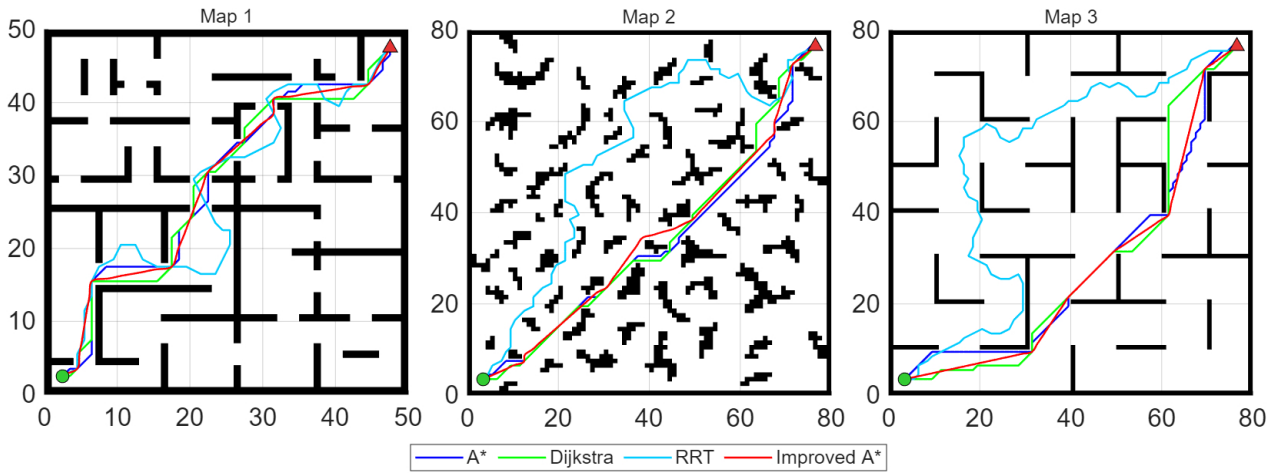


图 8. 四种算法在三张地图上的路径规划结果。

表 IV
代表性参数敏感性结果

Levels	Map	Best cost	Path Mean/std.dev	Run Time (ms)	Success rate (%)	Levels	Map	Best cost	Path Mean/std.dev	Run Time (ms)	Success rate (%)
	Map 1	**351.44**	**351.44/0.00**	**9.1**	**100**		Map 1	**351.44**	**351.44/0.00**	24.9	**100**
1/3/3/4/2	Map 2	**620.15**	620.34/0.44	**148.0**	80	3/3/4/4/2	Map 2	**620.15**	**620.15/0.00**	309.0	**100**
	kroB100	**22141.00**	22145.03/14.66	**250.5**	92.5		kroB100	**22141.00**	22142.45/9.17	350.1	97.5
	Map 1	**351.44**	**351.44/0.00**	15.1	**100**		Map 1	**351.44**	**351.44/0.00**	28.1	**100**
2/3/3/4/2	Map 2	**620.15**	620.27/0.29	204.7	85	3/3/5/4/2	Map 2	**620.15**	**620.15/0.00**	356.3	**100**
	kroB100	**22141.00**	22143.90/12.80	254.8	95		kroB100	**22141.00**	**22141.00/0.00**	407.1	**100**
	Map 1	**351.44**	**351.44/0.00**	27.2	**100**		Map 1	**351.44**	**351.44/0.00**	32.1	**100**
3/3/3/4/2	Map 2	**620.15**	**620.15/0.00**	241.5	**100**	3/3/3/1/2	Map 2	**620.15**	**620.15/0.00**	355.5	**100**
	kroB100	**22141.00**	22142.45/9.17	347.9	97.5		kroB100	**22141.00**	**22141.00/0.00**	474.8	**100**
	Map 1	**351.44**	**351.44/0.00**	40.4	**100**		Map 1	**351.44**	**351.44/0.00**	40.0	**100**
4/3/3/4/2	Map 2	**620.15**	**620.15/0.00**	371.1	**100**	3/3/3/2/2	Map 2	**620.15**	620.17/0.13	344.4	97.5
	kroB100	**22141.00**	**22141.00/0.00**	480.5	**100**		kroB100	**22141.00**	**22141.00/0.00**	421.5	**100**
	Map 1	**351.44**	**351.44/0.00**	52.7	**100**		Map 1	**351.44**	**351.44/0.00**	27.5	**100**
5/3/3/4/2	Map 2	**620.15**	**620.15/0.00**	336.3	**100**	3/3/3/3/2	Map 2	**620.15**	**620.15/0.00**	272.8	**100**
	kroB100	**22141.00**	22142.45/9.17	628.7	97.5		kroB100	**22141.00**	22142.45/9.17	357.8	97.5
	Map 1	**351.44**	**351.44/0.00**	37.8	**100**		Map 1	**351.44**	**351.44/0.00**	21.7	**100**
3/1/3/4/2	Map 2	**620.15**	620.17/0.13	426.5	97.5	3/3/3/5/2	Map 2	**620.15**	620.21/0.22	211.6	92.5
	kroB100	**22141.00**	**22141.00/0.00**	548.4	**100**		kroB100	**22141.00**	22145.35/15.47	342.0	92.5
	Map 1	**351.44**	**351.44/0.00**	27.2	**100**		Map 1	**351.44**	**351.44/0.00**	37.8	**100**
3/2/3/4/2	Map 2	**620.15**	**620.15/0.00**	311.8	**100**	3/3/3/4/1	Map 2	**620.15**	620.17/0.13	217.8	97.5
	kroB100	**22141.00**	**22141.00/0.00**	475.7	**100**		kroB100	**22141.00**	22193.30/37.45	786.8	30
	Map 1	**351.44**	**351.44/0.00**	30.5	**100**		Map 1	**351.44**	**351.44/0.00**	33.8	**100**
3/4/3/4/2	Map 2	**620.15**	620.19/0.18	257.0	95	3/3/3/4/3	Map 2	**620.15**	**620.15/0.00**	243.2	**100**
	kroB100	**22141.00**	22143.90/12.80	296.7	95		kroB100	**22141.00**	22143.90/12.80	404.3	95
	Map 1	**351.44**	**351.44/0.00**	38.5	**100**		Map 1	**351.44**	**351.44/0.00**	33.5	**100**
3/5/3/4/2	Map 2	**620.15**	620.75/1.17	232.1	62.5	3/3/3/4/4	Map 2	**620.15**	620.17/0.13	286.0	97.5
	kroB100	**22141.00**	22180.55/42.06	351.7	50		kroB100	**22141.00**	22142.45/9.17	390.8	97.5
	Map 1	**351.44**	**351.44/0.00**	44.4	**100**		Map 1	**351.44**	**351.44/0.00**	28.3	**100**
3/3/1/4/2	Map 2	**620.15**	620.71/1.05	217.1	65	3/3/3/4/5	Map 2	**620.15**	620.19/0.18	287.7	95
	kroB100	**22141.00**	22157.05/35.39	365.4	80		kroB100	**22141.00**	22142.45/9.17	487.7	97.5
	Map 1	**351.44**	**351.44/0.00**	32.2	**100**						
3/3/2/4/2	Map 2	**620.15**	620.35/0.35	247.2	75						
	kroB100	**22141.00**	22143.40/10.83	294.5	95						

时，kroB100 甚至可以达到 100% 成功率，但平均耗时增至约 0.35–0.41 s，且 Map 2 在 $y_{\max} = 0.90$ 下出现轻微波动。换言之，提高 $y_{\max}$ 能得到更稳的收敛，却要付出更多 VND 计算；默认 $y_{\max} = 0.4$ 与 S 形调度

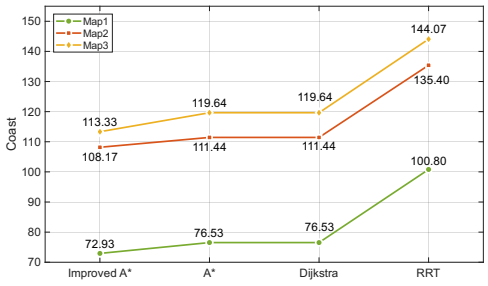


图 9. 四个算法在 Map 1、2、3 上的路径长度对比（折线图）。

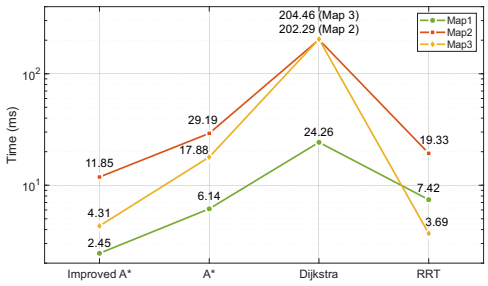


图 10. 四个算法在 Map 1、2、3 上的计算时间对比（折线图）。

配合，在迭代后半程集中精化，以较小代价获得接近上限的收益。

精英比率 r_{elite} 与候选列表大小 k 的敏感度相对较低。r_{elite} 在 0.1–0.9 之间变化时，kroB100 成功率多在 92.5%–100%，最优代价保持 22141；r_{elite} 过低时耗时增至约 0.47 s，过高则在 Map 2、kroB100 上出现小幅波动。k 在 9–60 范围内对代价几乎无影响，仅耗时随候选集变大而上升；但 $k = 3$ 是一个必须承认的下限例外：kroB100 成功率降至 30%、平均代价 22193.30、标准差 37.45，说明候选列表剪枝过狠会连同有希望的边交换一并滤掉，VND 难以完成有效精化。由此，k 宜取适中值：既能过滤远距离无效邻域，又不至于剪掉真正有用的改进。

还应指出，并不存在在所有指标上唯一占优的配置：更大种群或更高 $y_{\max}$ 有时能把 kroB100 成功率推到 100%，但时间几乎成倍增加；更低 q_0 同样更稳却更慢。因此评价须在“命中质量—稳定性—运行时间”的多目标口径下进行。默认参数在 Map 1、Map 2 上实现 100% 命中与零方差，在 kroB100 上以约 0.35 s 达到 97.5% 成功率，综合表现最均衡，故后续消融与算法对比实验均沿用该组设定。

D. 蚁群优化组件消融实验

为分离 MMAS-VND-CL 各机制的独立贡献，本组实验在保持其余设置不变的前提下，每次仅关闭一个机制，共设置六组，如表 V所示。每组在 Map 1、Map 2 与 kroB100 上运行 40 次，仍以最优代价、平均代价/标准差、耗时与成功率作为联合评价，实验的总结果见表 VI，同时还结合收敛曲线、代价分布与代价—时间对比交叉观察，如图 13–15。总体结论是：VND 与 MMAS 边界共同支撑“能否稳定拿到好解”，二者缺一不可；伪随机转移、S 形调度与候选列表则更偏向“多快拿到好解”，对解质量的边际影响相对有限，但决定着时间开销。需要提前说明的是，部分消融组在个别图上仍能摸到最优代价，甚至耗时更短，这类“看似更好”的现象并不与上述层次矛盾，下面结合机制展开。

所有地图上的完整消融结果见表 VI。消融过程可视化见图 13–15。

首先看 VND（组 3）。关闭局部搜索后性能全面崩溃：Map 2 成功率仅 2.5%，kroB100 为 0%，平均代价由 22142.45 升至 22566.98、标准差由 9.17 膨胀到 239.41，Map 1 成功率也从 100% 跌到 67.5%；图 14中该组箱体明显拉长，图 13显示其收敛曲线迟迟压不到最优平台。这与机制预期一致——构建阶段加信息素只能给出“还不错的顺序”，2-opt/重定位/交换所构成的 VND 才是把回路继续压向局部乃至全局最优的关键精化步骤；没有它，种群只能在较粗的解附近徘徊，方差与失败率同步上升。

其次是 MMAS 信息素边界（组 6）。去掉 $\tau_{\min}/\tau_{\max}$ 动态限幅后，Map 1 与 Map 2 成功率分别降至 30% 与 10%，kroB100 为 70%，平均代价与标准差均明显变差。原因是失去边界约束后，早期优势边上的信息素可被无限放大，搜索很快锁死在个别路径结构上，出现典型早熟；多次运行落入不同次优盆地，故“偶尔摸到最优、但多数失败”。值得注意的例外是：组 6 耗时反而最短。这并不表示算法更快，而是种群过早同质化后，自适应停止准则更早触发、无效迭代变少——以可靠性换来了表面上的速度。

再看两个“偏效率”的组件。去掉伪随机转移（组 2）后，在三张图上成功率均不低于完整算法，kroB100 甚至达到 100% 且标准差为 0，但耗时几乎翻倍。这说明在当前问题规模下，纯轮盘探索仍能收敛到优解，而 q_0 式的受控贪心主要价值在于加速收敛、减少搜索预算，与第 V-C 节中相互印证。去掉 S 形调度（组 4）意味着局部搜索比例不再随迭代抬升，而是从最初迭代时局部搜索比例就直接达到设定最大值，这导致质量仅轻微退化，但 Map 1 上耗时降至 17.5 ms，说明早期与后

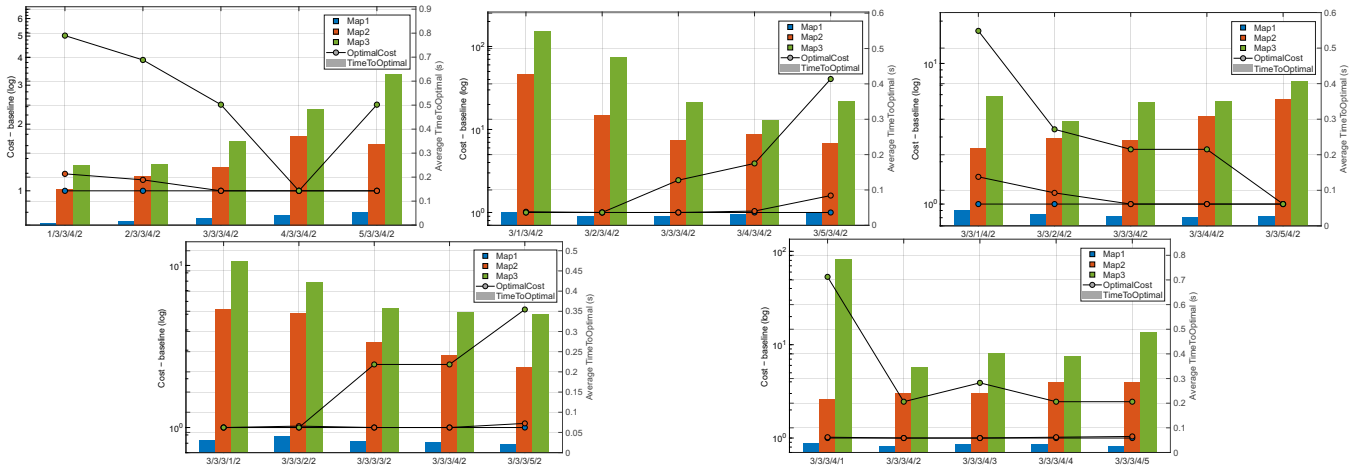


图 11. 各参数水平在 Map 1、Map 2、kroB100 上的代价与计算时间对比。

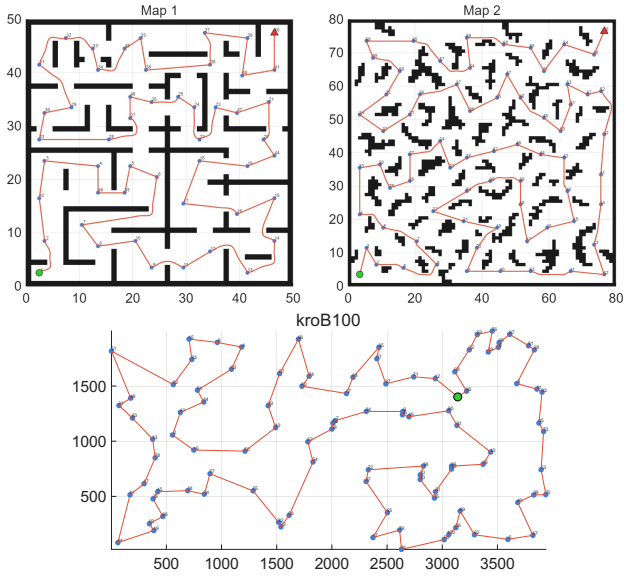


图 12. 默认参数设置下算法在 Map 1、Map 2、kroB100 上规划的路径图。

表 V
消融实验分组

组别	描述	禁用机制
1	完整算法	无
2	无伪随机	ACS 伪随机转移规则
3	无 VND	VND 局部搜索
4	无 S 曲线	渐进局部搜索调度
5	无候选列表	候选列表加速
6	无 MMAS	MMAS 信息素边界

期局部搜索比例保持相同时的确能省时，但也使早期蚂蚁的探索性有所下降，影响了算法的稳定性与最优解命中率。去掉候选列表（组 5）的解质量与完整算法几乎一致，仅耗时有所增加，符合其“不改变邻域最优性、

表 VI
消融实验结果

Map	Group	Best cost	Path Mean/std.dev	Run Time (ms)	Success rate (%)
Map 1	1	**351.44**	**351.44/0.00**	27.2	**100**
	2	**351.44**	**351.44/0.00**	32.4	**100**
	3	**351.44**	352.57/2.27	133.9	67.5
	4	**351.44**	**351.44/0.00**	**17.5**	**100**
	5	**351.44**	**351.44/0.00**	32.5	**100**
	6	**351.44**	359.64/12.48	21.8	30
Map 2	1	**620.15**	**620.15/0.00**	241.5	**100**
	2	**620.15**	**620.15/0.00**	415.7	**100**
	3	620.96	632.61/8.77	569.5	2.5
	4	**620.15**	620.17/0.13	349.4	97.5
	5	**620.15**	**620.15/0.00**	332.5	**100**
	6	**620.15**	626.12/6.12	**85.4**	10
kroB100	1	**22141.00**	22142.45/9.17	347.9	97.5
	2	**22141.00**	**22141.00/0.00**	731.3	**100**
	3	22279.00	22566.98/239.41	1473.9	0
	4	**22141.00**	22146.80/17.62	657.0	90
	5	**22141.00**	22143.90/12.80	426.0	95
	6	**22141.00**	22166.65/47.49	**248.4**	70

只减少无效评估”的定位——它不负责逃逸局部最优，只负责让 VND 跑得更便宜。

综合表 VI与图 15可以得到清晰的层次：VND 决定“能不能稳定逼近最优”，MMAS 边界决定“种群会不会过早锁死”，二者构成解质量的两大支柱；伪随机转移、S 形调度与候选列表主要调节收敛速度与计算开销，去掉它们未必搜不到好解，却会变慢或在中大型实例上损失稳定性。完整算法在 Map 1、Map 2 上实现 100% 命中，在 kroB100 上以约 0.35 s 取得 97.5% 成功率与较小方差，是质量、稳定性与时间的均衡点。由此，后续算法对比实验均以组 1 的完整配置为准；若应用场景更看重绝对成功率而非耗时，可适当加强后期局

部搜索比例或延长停滞上限，作为可调的改进空间。

E. 不同旅行商求解器的多目标点遍历对比

在固定前端（改进 A* 代价矩阵 + 虚拟节点开放编码）的前提下，本组实验更换后端旅行商求解器，以检验完整多目标点遍历框架中“排序模块”的实际贡献。参与对比的四种求解器为：本文 MMAS-VND-CL、标准蚁群系统（ACO）、遗传算法（GA）与模拟退火（SA），参数见表 VII；每种组合在 Map 1、Map 2 与 kroB100 上独立运行 40 次。结论可以概括为：在统一几何代价接口下，MMAS-VND-CL 在解质量、命中率与稳定性上全面领先，且时间开销与轻量基线处于同一量级；ACO、SA 有时更快，但从未稳定触及已知最优代价，GA 则在质量与时间上均明显落后。下面按指标—条件—机制展开，并如实讨论个别“更快却不更好”的情形。

表 VII
对比旅行商求解器的参数设置

Algorithm	Main parameters
MMAS-VND-CL	$n_{\text{ants}} = 40$；$\alpha = 1$；$\beta = 2$； $\rho = 0.25$；$Q = 225$；$\tau_0 = 0.1$； $q_0 = 0.45$；$y_{\max} = 0.4$； Local search transition interval= $[0, 30]$； 比 $r_{\text{elite}} = 0.7$；$k = 9$
ACO	$n_{\text{ants}} = 40$；$\alpha = 1$；$\beta = 2$；$\rho = 0.25$； $Q = 225$；$\tau_0 = 0.1$；Iter = 100
GA	Pop = 100；Iter = 1500； $P_c = 0.8$；$P_m = 0.25$； Number of elites= 10
SA	$T_0 = 150$；$\alpha = 0.996$； $L = 10$；$T_{\text{end}} = 10^{-8}$

每个求解器的多目标点遍历性能见表 VIII。对应可视化结果见图 16–19。

表 VIII
多目标点遍历对比结果

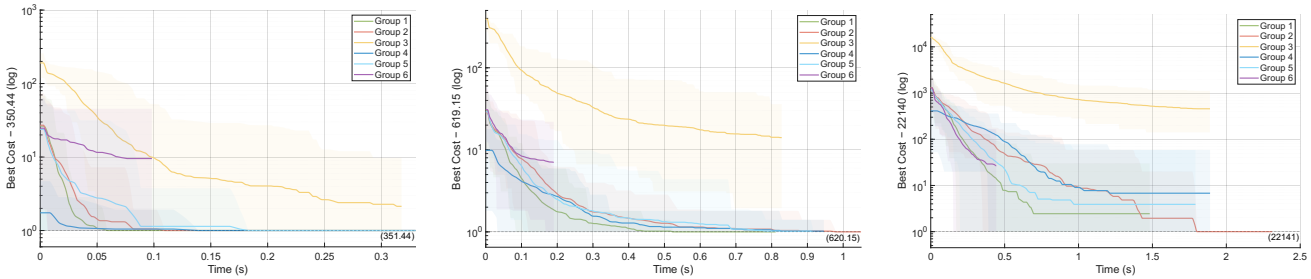
Map	Algorithm	Best cost	Path Mean/std.dev	Run Time (ms)	Success rate (%)
Map 1	MMAS-VND-CL	**351.44**	**351.44/0.00**	**27.2**	**100**
	ACO	359.32	388.65/15.57	56.4	0
	GA	404.45	485.16/44.26	3000.7	0
	SA	359.98	390.35/13.94	390.2	0
Map 2	MMAS-VND-CL	**620.15**	**620.15/0.00**	241.5	**100**
	ACO	638.68	685.75/18.91	**130.4**	0
	GA	698.41	851.25/46.56	3842.4	0
	SA	664.56	692.00/66.33	732.1	0
kroB100	MMAS-VND-CL	**22141.00**	**22142.45/9.17**	347.9	**97.5**
	ACO	23359.00	24435.83/559.30	330.0	0
	GA	33271.00	37360.65/2680.12	4988.2	0
	SA	23574.00	24918.35/752.52	**321.8**	0

先看解质量与命中率。表 VIII显示，MMAS-VND-CL 在三张图上均取得最低的最优代价和平均代价，并且是唯一能稳定命中已知最优的求解器：Map 1 与 Map 2 上 40 次运行全部达到 351.44 与 620.15，kroB100 上以 97.5% 命中 22141、平均 22142.45。对于其他三个对比算法，它们的成功率均为 0%——它们或许能在个别运行中接近较优值，却从未触及同一最优代价。差距在 kroB100 上最为明显：最优基线 ACO 的平均代价 24435.83 较本文高约 10.4%，SA 高约 12.6%，GA 平均 37360.65 则高出约 68.7%。图 19中基线路径的迂回与交叉明显多于本文方法，图 17的箱体也更长、须更散，说明其“偶尔并不太差、整体却系统性偏长”。

再看稳定性。MMAS-VND-CL 在 kroB100 上标准差仅 9.17，而 ACO、SA、GA 分别为 559.30、752.52 与 2680.12，相差约两个数量级；Map 2 上 SA 的 66.33 与本文的 0.00 对比同样鲜明。结合机制可以理解：虚拟节点编码使起终点固定且完整参与信息素更新，MMAS 边界避免早期锁死，候选列表加速的 VND 则在每一代对精英回路做强力精化，三者叠加把运行之间的波动压到很低；缺少 VND 与边界控制的标准 ACO、SA 只能依赖构建/邻域的随机扰动，路径质量随初始化散布较宽。

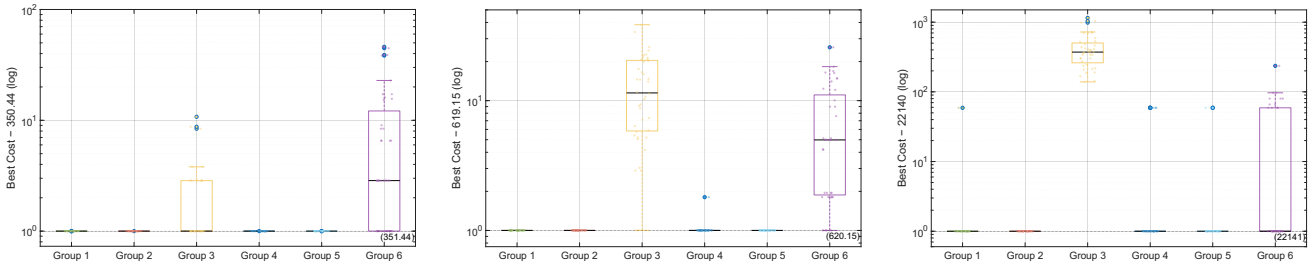
然后看时间，这里需要客观让步。表 VIII中，Map 2 上 ACO 以 130.4 ms 快于本文的 241.5 ms，kroB100 上 SA 与 ACO 也略快于 347.9 ms。原因很直接：这些基线没有大规模 VND 邻域搜索，迭代一次更便宜；本文把部分预算花在局部搜索上，单次迭代更重。但若把“单位时间换来的解质量”算进去，结论会反过来——基线在更短或相近时间内给出的平均代价高出 10% 以上且命中率为 0；GA 更是全面落后，三图耗时达数秒，代价仍最差，说明其变长染色体交叉在路径型表示上缺乏针对性，搜索浪费明显。因此，不能只比实际耗时，还应看时间—代价折中，如图 18所示：本文方法处于“低代价—适中时间”的有效前沿附近。

最后作机制归因与收束。在统一的 JPS 代价矩阵上，组合层的差别被单独放大：虚拟节点开放编码保证首尾约束不被搜索破坏；MMAS 信息素边界维持多样性；ACS 式伪随机加速收敛；候选列表 VND 负责把回路压向近优；S 形调度与自适应停止则控制何时、为谁支付精化成本。实验表明，真正拉开与基线差距的是



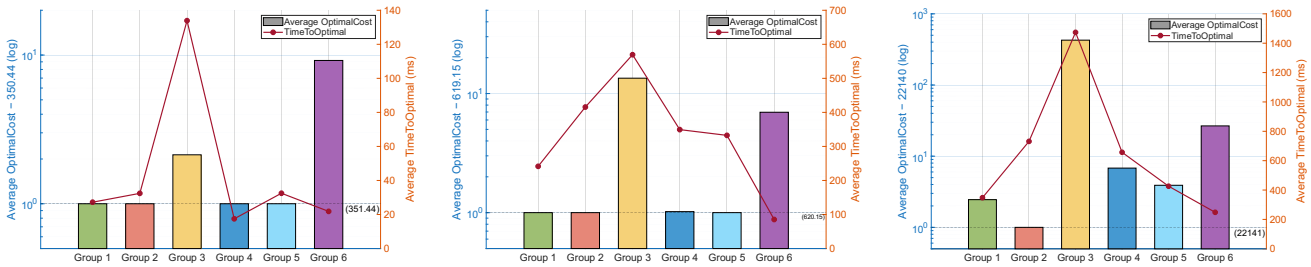
(a) Map 1 (b) Map 2 (c) kroB100

图 13. 六个消融组在三张地图上 40 次运行的平均迭代次数-代价收敛曲线。



(a) Map 1 (b) Map 2 (c) kroB100

图 14. 六个消融组在三张地图上 40 次运行的代价分布（箱线图）。



(a) Map 1 (b) Map 2 (c) kroB100

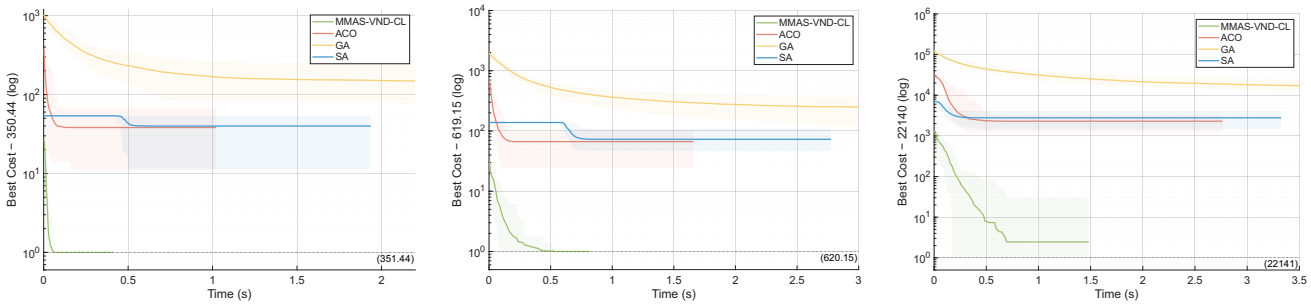
图 15. 六个消融组在三张地图上的平均代价与平均计算时间对比。

“稳定命中最优”这一能力，而不仅是某一次的最好值。当然也应承认：若应用只要求快速得到任意可行访问序，ACO 或 SA 仍有低延迟优势；若目标点规模继续增大，VND 的计算占比会上升，届时需要进一步加强候选列表压缩或分层局部搜索。综合质量、可靠性与可接受开销，MMAS-VND-CL 更适合作为障碍环境多目标点遍历的排序模块。

除上述仿真地图外，本文还选取了一块真实卫星场景做验证。从 Google 地图截取该区域后，将其转换为二进制栅格（黑色为障碍物、白色为可通行空地），并对手工处理后的地图做了适度优化，以更贴近导航需求。图 **??**自左至右依次为：原始卫星图（红框为实验场地）、场地的二进制地图及目标点配置（黄点为中间目标，绿点为起点，红三角为终点），以及在真实地图背景上显示的规划结果。可以看到，算法能够在真实场景中生成穿越可通行区域、合理连接各目标点的连续路径，表明所提方法可方便地由仿真栅格迁移到由真实影像构建的导航环境中。

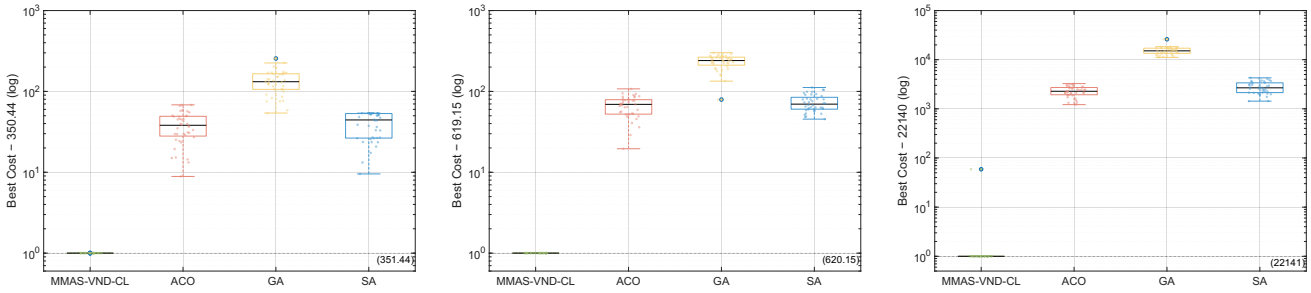
VI. 结论

本文提出了一种面向障碍环境多目标点遍历路径规划的协同优化框架，将基于跳点搜索的改进 A* 全局规划器与求解开放旅行商问题的 MMAS-VND-CL 蚁群优化算法相集成。该框架弥补了传统旅行商问题研究中忽略障碍物代价与路径规划器不优化中间节点访问顺序的问题。具体而言，利用本文提出的改进 A* 算法



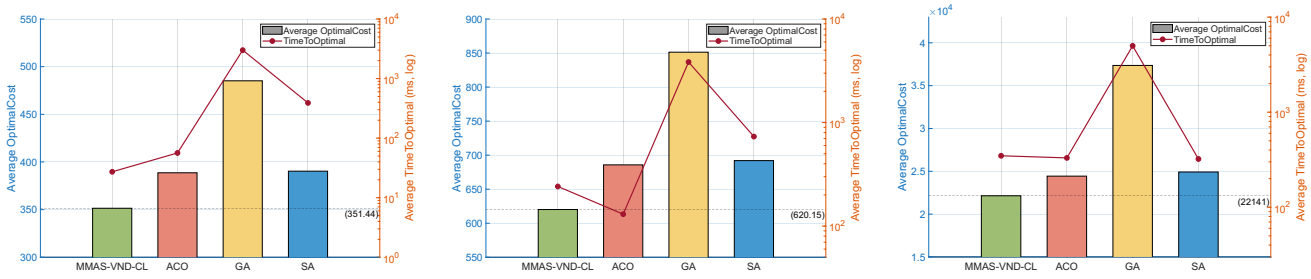
(a) Map 1 (b) Map 2 (c) kroB100

图 16. 四个旅行商求解器在三张地图上 40 次运行的平均时间-代价收敛曲线。



(a) Map 1 (b) Map 2 (c) kroB100

图 17. 四个旅行商求解器在三张地图上 40 次运行的代价分布（箱线图）。



(a) Map 1 (b) Map 2 (c) kroB100

图 18. 四个旅行商求解器在三张地图上的平均代价与平均计算时间对比。

将具有障碍信息的路径成本编码为代价矩阵，MMAS-VND-CL 算法基于该矩阵确定节点访问顺序，从而将几何推理与组合优化统一到同一管线中。

在路径规划层面，跳点搜索借助栅格对称性剪枝冗余节点，安全感知简化与弧长参数化样条平滑进一步压缩航点并生成连续可跟踪轨迹。在排序层面，虚拟节点法把开放问题转化为闭环问题并使起终点参与信息素更新，MMAS 动态边界抑制早熟，ACS 式伪随机转移平衡利用与探索，候选列表加速的 VND 把局部搜索复杂度由 $O(K^2)$ 降至 $O(K \cdot k)$，S 形调度与自适应停止准则则控制精化时机并避免无效迭代。

实验验证了上述设计。改进 A* 相较传统 A* 最多减少约 5.3% 的路径长度，在 Map 2 上将规划时间由 202.29 ms 降至 11.85 ms，降低了约 94.1% 的算法耗时并给出更短路径。对于 MMAS-VND-CL 算法在敏感性分析中表明，默认参数处于均衡区，其中 q_0 与 $y_{\max}$ 最为敏感；消融实验显示 VND 与 MMAS 边界是解质量的两大支柱，伪随机转移、S 形调度与候选列表主要改善收敛速度与计算效率。在 kroB100 上，MMAS-VND-CL 以 97.5% 的命中率达到最优代价，较最优基线 ACO 改善约 10.4%，标准差比 GA 小一个数量级以上。

后续工作将从三方面展开：面向动态障碍的在线重规划与三维地形扩展；候选列表规模等超参数的自适应

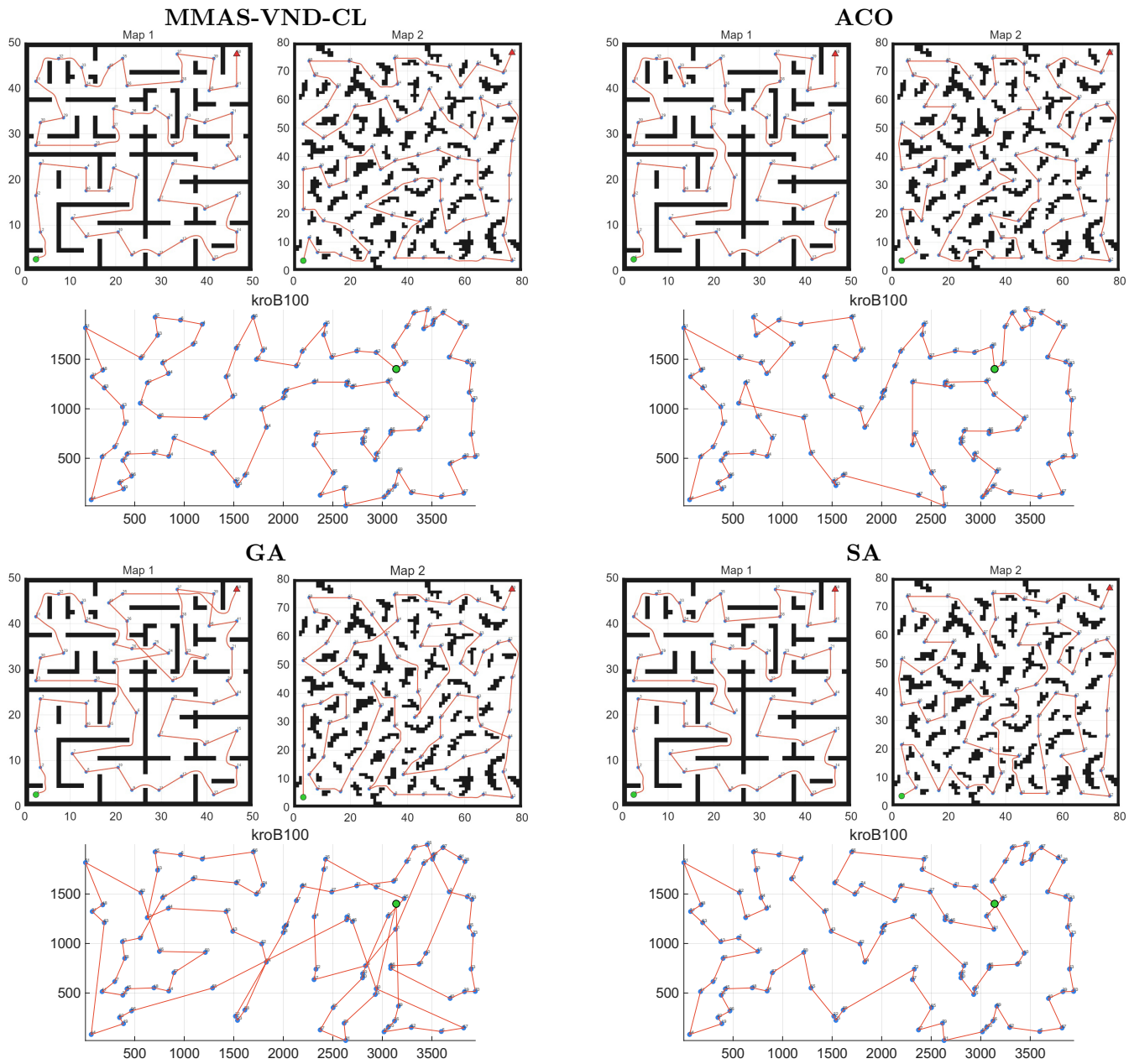


图 19. 各旅行商求解器在 Map 1、Map 2、kroB100 上规划的最优代价路径图。

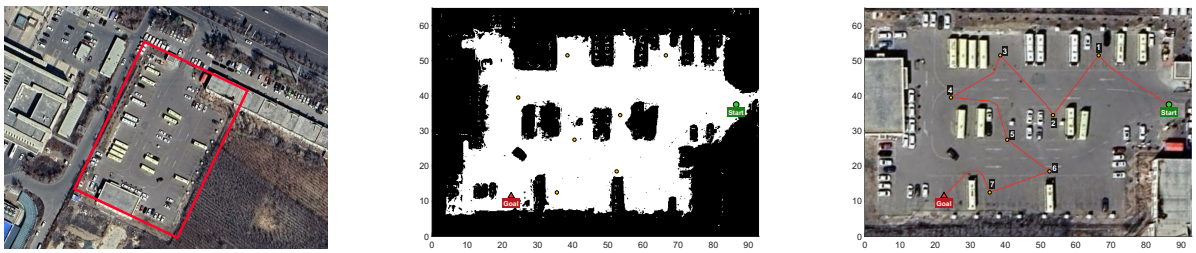


图 20. 真实卫星场景验证。自左至右：Google 地图截取区域（红框为实验场地）、转换后的二进制地图与目标点配置、真实地图背景上的规划路径。

或学习式调节；向多机器人协同遍历扩展，并以距离预言机近似或增量代价精化降低大规模代价矩阵的计算开销。

参考文献

[1] S. B. Liu et al., “SMUG planner: A safe multi-goal planner for mobile robots in challenging environments,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 9, pp. 5504–5511, 2023.

[2] H. T. T. Binh, “Fast marching firework method for multi-goal mobile robot path planning in complex obstacle maps,” *Intell. Service Robot.*, vol. 18, 2025, doi: 10.1007/s11370-025-00654-6.

[3] C.-G. Wu, Y.-C. Liang, H.-P. Lee, C. Lu, and C. Lin, “Solving constrained traveling salesman problems by genetic algorithms,” *Prog. Nat. Sci.*, vol. 14, no. 7, pp. 631–637, 2004.

[4] “A survey on approximability of traveling salesman problems using the TSP-T3CO definition scheme,” *Ann. Oper. Res.*, 2025, doi: 10.1007/s10479-025-06641-5.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[6] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959.

[7] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Dept. Comput. Sci., Iowa State Univ., Ames, IA, USA, Tech. Rep. 98-11, 1998.

[8] “A comprehensive review of improved A* path planning algorithms and their hybrid integrations,” *Automation*, vol. 6, p. 52, 2025.

[9] L. H. O. Rios and L. Chaimowicz, “A survey and classification of A* based best-first heuristic search algorithms,” in *Proc. 20th Brazilian Symp. Artif. Intell. (SBIA)*, 2010, pp. 253–262.

[10] M. Likhachev, G. Gordon, and S. Thrun, “ARA*: Anytime A* with provable bounds on sub-optimality,” in *Adv. Neural Inf. Process. Syst. (NIPS)*, 2003, pp. 767–774.

[11] D. Harabor and A. Grastien, “Online graph pruning for pathfinding on grid maps,” in *Proc. 25th AAAI Conf. Artif. Intell. (AAAI-11)*, 2011, pp. 1114–1119.

[12] D. Harabor and A. Grastien, “Improving jump point search,” in *Proc. 24th Int. Conf. Automated Planning Scheduling (ICAPS-14)*, 2014, pp. 128–135.

[13] “Improved JPS and DWA path planning algorithm for mobile robots,” *Expert Syst. Appl.*, 2026.

[14] “Research on mobile robot path planning based on an improved bidirectional jump point search algorithm,” *Electronics*, vol. 14, no. 8, p. 1669, 2025, doi: 10.3390/electronics14081669.

[15] “Adaptive weight optimization based jump point Theta* algorithm in mobile robot path planning for intricate environments,” *Discover Appl. Sci.*, vol. 7, p. 11, 2025, doi: 10.1007/s42452-025-07932-z.

[16] “UGV path optimization in UAV-assisted environments using visibility-aware path simplification,” *J. Sens. Actuator Netw.*, vol. 15, no. 3, p. 41, 2025.

[17] “Enhancing the safety and smoothness of path planning through an integration of Dijkstra’s algorithm and piecewise cubic Bezier optimization,” *Expert Syst. Appl.*, 2025.

[18] “Adaptive A*/NSGA-II framework for multi-goal navigation of mobile robots,” *Meas. Sci. Technol.*, 2025, doi: 10.1088/1361-6501/ae9209.

[19] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[20] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[21] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, 1996.

[22] M. Dorigo and L. M. Gambardella, “Ant colony system: A cooperative learning approach to the traveling salesman problem,” *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, 1997.

[23] T. Stützle and H. H. Hoos, “MAX–MIN ant system,” *Future Gener. Comput. Syst.*, vol. 16, no. 8, pp. 889–914, 2000.

[24] P. Hansen and N. Mladenović, “Variable neighborhood search: Principles and applications,” *Eur. J. Oper. Res.*, vol. 130, no. 3, pp. 449–467, 2001.

[25] N. A. Kyriakakis, M. Marinaki, and Y. Marinakis, “A hybrid ant colony optimization–variable neighborhood descent approach for the cumulative capacitated vehicle routing problem,” *Comput. Oper. Res.*, vol. 134, p. 105397, 2021.

[26] “An improvement to the 2-opt heuristic algorithm for approximation of optimal TSP tour,” *Appl. Sci.*, vol. 13, no. 12, p. 7339, 2023, doi: 10.3390/app13127339.

[27] “Matrix-based ant colony system for traveling salesman problem,” in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, 2024.

[28] “nLKH-ACS: A niching Lin-Kernighan–Helsgaun-based ant colony system for multisolution traveling salesman problems,” *IEEE Trans. Evol. Comput.*, 2024.

[29] “CAACS: A carbon aware ant colony system,” *Sci. Rep.*, 2025.

[30] “Ant colony optimization for multiple traveling salesmen problem with revisitable cities,” in *Proc. IEEE*, 2024.

[31] B. Yao, Q. Hu, L. Zhang, and J. Tian, “Improved ant colony optimization for seafood product delivery routing problem,” *Promet–Traffic Transp.*, vol. 26, no. 1, 2014.

[32] B. Yao, C. Chen, X. Song, and X. Yang, “Fresh seafood delivery routing problem using an improved ant colony optimization,” *Ann. Oper. Res.*, vol. 273, no. 1–2, pp. 163–186, 2019, doi: 10.1007/s10479-017-2531-2.